

# **Ice Cream Manufacturer Database Management System**

**CPS510: Database Systems I**

**Assignment Number: 10**

**Azaan Cheema: 5011XXXXX**

**Maryam Ramzaan: 5011XXXXX**

**Daniyal Nadeem: 5007XXXXX**

# Table of Contents

|                                                            |              |
|------------------------------------------------------------|--------------|
| <b>Assignment 1:</b> Application Description               | <b>3-7</b>   |
| <b>Assignment 2:</b> ER Model                              | <b>8</b>     |
| <b>Assignment 3:</b> Schema Design                         | <b>9-16</b>  |
| <b>Assignment 4a:</b> Simple Queries                       | <b>16-20</b> |
| <b>Assignment 4b:</b> Designing views/Advanced Queries     | <b>20-28</b> |
| <b>Assignment 5:</b> Interesting Queries and UnixShell     | <b>28-41</b> |
| <b>Assignment 6:</b> Normalization/Functional Dependencies | <b>42</b>    |
| <b>Assignment 7:</b> Normalization /3NF                    | <b>43-48</b> |
| <b>Assignment 8:</b> Normalization 3NF/BCNF by Algorithm   | <b>48-52</b> |
| <b>Assignment 9:</b> User Interface of DBMS                | <b>53-69</b> |
| <b>Assignment 10:</b> Relational Algebra                   | <b>60-72</b> |
| <b>Conclusion</b>                                          | <b>72</b>    |
| <b>References</b>                                          | <b>73</b>    |

## Assignment 1: Application Description

### Introduction

The world of ice cream manufacturing is highly competitive. Maintaining clean and accurate records for business operations are crucial for ensuring efficiency and accuracy, since the company can deliver high-quality products to consumers. A company, such as an Ice Cream Manufacturer, would have many different operations including multiple stores to supply to. These suppliers provide the raw ingredients, employees, ingredients and allergens, and invoices. For a large operation such as this, a database management system would provide a structured framework that would help with keeping track of all the daily logs, and it would help the manufacturer optimize their operations, reduce errors and redundancies, and ultimately help with the decision making progress in regards to the companies' future. The goal of this project is to understand how relational algebra can help with designing a database while ensuring smooth daily operations within a company.

### Importance of the Database

A structured database plays a very important role in managing the complicated processes and transactions of any organization. Easy access to data becomes more readily available, data consistency can improve, and the overall operational efficiency and reliability will increase due to more accessible and organized knowledge [1].

The database can help with many factors, including:

#### Efficiency

- If the data is in structured storage, then the data retrieval is quick and easy. With easy data retrieval, data analytics can then be performed. [2]

#### Data Integrity

- If the key entities are linked, then the database will reduce redundancies and ensure accuracy. [2]

#### Decision Making

- With a structured and accurate database, access to data is available at any given time. Should a decision be required then data can be analyzed at any given time, and an up-to-date decision can be made. [2]

### Maintenance & Scalability

- Automated processes can be set up if the database is connected to the relevant platform. Ordering stock, inventory management, payments & invoices, and order fulfillment can all be handled easily. Should the businesses need to expand, then the information about the new stores or flavors can be added to the database only, and the system will handle the rest. [2]

## Importance of an Ice Cream Manufacturer

As various products in the modern world are created and sold, ice cream requires a database to keep track of all the items needed to make and sell the dessert to the desired audience. A database is important for large manufacturers that provide ice cream and sell products to thousands of other companies and customers. For example, if the manufacturer does not keep track of what ingredients are in the inventory, as well as which customers received their products, the company could lose millions of dollars worth of profit in a day. Relationships between suppliers and customers would be strained, which is even worse. In order to keep up with all the information regarding the manufacturer, and to stay up to date to maximize business, maintaining a database is more efficient. However, to create that database, there are important entities that need to be recognized and individuals who need to be updated and contacted regularly. Relationships between entities and other individuals are crucial to forming the database and are often shown on an ER (Entity Relationship) diagram.

## Relationships between the Entities

An ice cream manufacturer needs to manufacture the ice cream by creating flavors, purchasing the ingredients, as well as having access to all the information regarding products from suppliers to prepare allergy warnings, nutrition labels, and so on. This can all be categorized as a Flavour entity on the database. Once the ice cream is made, it will need to be labeled and the quantity of ice cream needs to be recorded as part of the inventory; this is another entity called Ice Cream. The entity, Ice Cream, depends on the flavor, because if there are no ingredients or no more flavors left, there will be no ice cream made, furthermore, each flavor will get its own Ice Cream ID once it is made, hence it has a one to one relationship. After the ice

cream is made, it can be ordered by customers, and employees help this happen. There are many employees, and each employee handles one order at a time, creating a one-to-many relationship between these entities. There is also an entity to record the customer's information, such as an address, which is a weak entity, as an address without a customer is meaningless. Furthermore, each order has a customer, and an invoice for each order. If there is no order, there is no invoice, which makes the invoice a weak entity, and the order a strong entity. There are some functions and information that the invoice and customer have in common, such as payment information.

To manage a large-scale company that manufactures ice cream, a database can be used to organize and record information about sales, customers, and product inventory. Important aspects of the company to consider are entities in the database. Entities include Flavor, Ice Cream, Order, Invoice, Customer, Address and Payment. Each entity is related to each other through the order in which tasks are performed and some entities depend on each other. These relationships can be captured on an ER diagram.

### Database System Entities and Attributes

| Entity    | Attributes                                                                                                                                                                                                    |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customer  | <ul style="list-style-type: none"> <li>• Customer ID</li> <li>• Customer Name</li> <li>• AddressID</li> <li>• Phone Number</li> <li>• Email</li> <li>• Payment ID</li> <li>• Order ID</li> </ul>              |
| Employee  | <ul style="list-style-type: none"> <li>• Employee ID</li> <li>• Employee Name</li> <li>• Birth Date</li> <li>• Role</li> <li>• Email</li> <li>• Phone Number</li> <li>• Salary</li> <li>• Order ID</li> </ul> |
| Ice Cream | <ul style="list-style-type: none"> <li>• Ice Cream ID</li> <li>• Price Per Unit</li> <li>• Stock Quantity</li> <li>• Flavour ID</li> </ul>                                                                    |
| Flavour   | <ul style="list-style-type: none"> <li>• Flavour ID</li> <li>• Flavour Types</li> <li>• Allergy Information</li> </ul>                                                                                        |

|                     |                                                                                                                                                                                                                                                     |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                     | <ul style="list-style-type: none"> <li>• Availability</li> <li>• Flavour Description</li> <li>• Ingredients</li> </ul>                                                                                                                              |
| Invoice             | <ul style="list-style-type: none"> <li>• Invoice ID</li> <li>• Order ID</li> <li>• Invoice Date</li> <li>• Invoice Total Amount</li> <li>• Invoice Status</li> <li>• Customer ID</li> <li>• Payment Information ID</li> </ul>                       |
| Address             | <ul style="list-style-type: none"> <li>• Address ID</li> <li>• Customer ID</li> <li>• Location</li> <li>• City</li> <li>• Province</li> <li>• Country</li> <li>• Postal Code</li> </ul>                                                             |
| Payment Information | <ul style="list-style-type: none"> <li>• Payment ID</li> <li>• Method of Payment</li> <li>• Payment Date</li> <li>• Payment Status</li> <li>• Payment Amount</li> <li>• Customer ID</li> <li>• Invoice ID</li> </ul>                                |
| Orders              | <ul style="list-style-type: none"> <li>• Order ID</li> <li>• Customer ID</li> <li>• Order Date</li> <li>• Total Order</li> <li>• Order Status</li> <li>• Payment ID</li> <li>• Employee ID</li> <li>• Invoice ID</li> <li>• Ice Cream ID</li> </ul> |

## Functionality

|                     |            |                     |        |
|---------------------|------------|---------------------|--------|
| Customer            | places()   | Order               | 1 to N |
| Order               | contains() | Ice Cream           | 1 to N |
| Ice cream           | has()      | Flavour             | 1 to 1 |
| Order               | has()      | Invoice             | 1 to 1 |
| Invoice             | has()      | Payment Information | 1 to N |
| Invoice             | has()      | Customer            | M to N |
| Payment Information | has()      | Customer            | 1 to 1 |
| Customer            | has()      | Address             | 1 to 1 |

## **Assignment 2: ER Model**

The ER Model is included in the next page of the report.



## Assignment 3: Schema Design

### Schema Diagram Codes:

```
DROP TABLE has;  
DROP TABLE Address;  
DROP TABLE Invoice;  
DROP TABLE PaymentInformation;  
DROP TABLE Order_IceCream;  
DROP TABLE Employee;  
DROP TABLE Customer;  
DROP TABLE IceCream;  
DROP TABLE Flavour;
```

```
CREATE TABLE Flavour (  
    Flavour_ID NUMBER PRIMARY KEY,  
    Flavour_Type VARCHAR2(50) NOT NULL UNIQUE,  
    Flavour_AllergyInfo VARCHAR2(225),  
    Flavour_Available VARCHAR2(10) NOT NULL,  
    Flavour_Description VARCHAR2(255),  
    Flavour_Ingredients VARCHAR2(255)  
);
```

```
CREATE TABLE IceCream (  
    Icecream_ID NUMBER PRIMARY KEY,  
    Icecream_FlavourID NUMBER UNIQUE,  
    Icecream_PricePerUnit DECIMAL(10, 2) DEFAULT 0.0,  
    Icecream_StockQuantity NUMBER(10) DEFAULT 0,  
    FOREIGN KEY (Icecream_FlavourID) REFERENCES Flavour(Flavour_ID)  
);
```

-- Strong Entity: Customer

-- Stores customer details

```
CREATE TABLE Customer(  
    Customer_ID VARCHAR2(20) NOT NULL,  
    Customer_Name VARCHAR2(50) NOT NULL,  
    Customer_Address_ID VARCHAR2(50) NOT NULL,  
    Customer_Phone_Number VARCHAR2(15) NOT NULL,  
    Customer_Email VARCHAR2(30) NOT NULL,  
    Customer_Payment_ID VARCHAR2(5) NOT NULL,
```

```

PRIMARY KEY (Customer_ID)
);

-- Strong Entity: Employee
-- Holds personal information about the employee
CREATE TABLE Employee(
    Employee_ID VARCHAR2(20) NOT NULL,
    Employee_Name VARCHAR2(50) NOT NULL,
    Employee_Birth_Date VARCHAR2(15) NOT NULL,
    Employee_Role VARCHAR2(60) NOT NULL,
    Employee_Email VARCHAR2(30) NOT NULL,
    Employee_Phone_Number VARCHAR2(15) NOT NULL,
    Employee_Salary DECIMAL(8,2),
    Employee_Order_ID VARCHAR2(15),
    PRIMARY KEY (Employee_ID)
);

-- Strong Entity: Order (Order_IceCream)
-- Holds the orders that the customer placed
CREATE TABLE Order_IceCream(
    Order_ID VARCHAR2(20) NOT NULL,
    Order_Employee_ID VARCHAR2(50) NOT NULL UNIQUE,
    Order_IceCream_ID NUMBER NOT NULL UNIQUE,
    Order_Payment_ID VARCHAR2(10) NOT NULL UNIQUE,
    Order_Order_Status VARCHAR2(20) NOT NULL,
    Order_Total_Order DECIMAL(8,2) NOT NULL,
    Order_Invoice_ID VARCHAR2(15) NOT NULL UNIQUE,
    Order_Date DATE NOT NULL,
    Order_Customer_ID VARCHAR2(10) NOT NULL UNIQUE,
    PRIMARY KEY (Order_ID)
);

CREATE TABLE PaymentInformation (
    Payment_ID VARCHAR2(50) NOT NULL,
    Method_of_Payment VARCHAR2(50) NOT NULL,
    Payment_Date DATE NOT NULL,
    Payment_Status VARCHAR2(50) NOT NULL,
    Payment_Amount DECIMAL(10, 2) NOT NULL,
    Customer_ID VARCHAR2(50) UNIQUE,
    Invoice_ID VARCHAR2(50) UNIQUE,

```

```

PRIMARY KEY (Payment_ID)
);

-- Weak Entity: Invoice
-- Helps store the bill details which the customer placed an order
CREATE TABLE Invoice (
    Invoice_ID VARCHAR2(20) NOT NULL,
    Invoice_Order_ID VARCHAR2(20) NOT NULL UNIQUE,
    Invoice_Date VARCHAR2(40) NOT NULL,
    Invoice_Total_Amount DECIMAL(8,2) NOT NULL,
    Invoice_Status VARCHAR2(40) NOT NULL,
    Invoice_Customer_ID VARCHAR2(20) NOT NULL UNIQUE,
    Invoice_Payment_Information_ID VARCHAR2(30) NOT NULL UNIQUE,
    FOREIGN KEY (Invoice_Order_ID) REFERENCES Order_IceCream(Order_ID),
    FOREIGN KEY (Invoice_Customer_ID) REFERENCES Customer(Customer_ID),
    PRIMARY KEY (Invoice_ID)
);

CREATE TABLE Address (
    Address_CustomerID VARCHAR2(50) NOT NULL,
    Address_Province VARCHAR2(255),
    Address_City VARCHAR2(255),
    Address_Location VARCHAR2(255),
    Address_ID VARCHAR2(50) NOT NULL,
    Address_Country VARCHAR2(255),
    Address_PostalCode VARCHAR2(255),
    PRIMARY KEY (Address_CustomerID, Address_ID),
    FOREIGN KEY (Address_CustomerID) REFERENCES Customer(Customer_ID)
);

-- Relationship: between Customer and Invoice
CREATE TABLE has (
    Customer_ID VARCHAR2(20) NOT NULL,
    Invoice_ID VARCHAR2(20) NOT NULL,
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
    FOREIGN KEY (Invoice_ID) REFERENCES Invoice(Invoice_ID),
    PRIMARY KEY (Customer_ID, Invoice_ID)
);

-- Populate Tables

```

```
--Flavour(ID,TYPE,ALLERGY,AVAILABLE, DESCRIPTION, INGREDIENTS)
INSERT INTO Flavour VALUES(1, 'Mint Chocolate Chip', 'NO Nuts', 'Y', 'Fresh minty ice
cream with chocolate chips', 'mint, cocoa, ice cream');
INSERT INTO Flavour VALUES(2, 'Chocolate', 'May contain Hazel nuts', 'Y', 'Rich chocolate
ice cream made from high-quality cocoa', 'cocoa, cream, sugar');
INSERT INTO Flavour VALUES(3, 'Vanilla', 'No nuts', 'N', 'Classic vanilla ice cream with a
smooth texture', 'vanilla extract, cream, sugar');
INSERT INTO Flavour VALUES(4, 'Strawberry', 'No nuts', 'N', 'Fresh strawberry ice cream
made with real strawberries', 'strawberries, cream, sugar');
INSERT INTO Flavour VALUES(5, 'Cookie Dough', 'May contain nuts', 'Y', 'Delicious ice
cream with chunks of cookie dough and chocolate chips', 'cookie dough, chocolate chips,
cream');
```

```
--F
INSERT INTO IceCream VALUES(14, 1, 2.29, 1000);
INSERT INTO IceCream VALUES(15, 2, 2.49, 800);
INSERT INTO IceCream VALUES(16, 3, 2.19, 500);
INSERT INTO IceCream VALUES(17, 4, 2.39, 700);
INSERT INTO IceCream VALUES(18, 5, 2.59, 600);
```

```
-- Customer(Customer_ID, Customer_Name, Customer_Address, Customer_Phone_Number,
Customer_Email, Customer_Payment_ID)
INSERT INTO Customer VALUES('AA1', 'Azaan', '341 Yonge St, Toronto, ON M4B71',
'647-899-1234', 'azaan@gmail.com', 'P1');
INSERT INTO Customer VALUES('AA2', 'Maryam', '15 Wallingford Rd, North York, ON
M3G2V1', '647-343-1567', 'maryam@gmail.com', 'P2');
INSERT INTO Customer VALUES('AA3', 'Dan', '27 Kings College Cir, Toronto, ON M5K1A1',
'647-256-1273', 'dan@gmail.com', 'P3');
INSERT INTO Customer VALUES('AA4', 'Susan', '300 Borough Dr, Scarborough, ON
M1O4P5', '647-999-1415', 'susan@gmail.com', 'P4');
INSERT INTO Customer VALUES('AA5', 'Ali', '245 Church St, Toronto, ON M1T6W1',
'647-766-1322', 'ali@gmail.com', 'P5');
```

```
-- Employee(Employee_ID, Employee_Name, Employee_Birth_Date, Employee_Role,
Employee_Email, Employee_Phone_Number, Employee_Salary, Employee_Order_ID)
INSERT INTO Employee VALUES ('EM1', 'Vaishnavi', '02/02/1999', 'Production Operator',
'vaishnavi@gmail.com', '647-222-5555', 60000.50, 'R1');
INSERT INTO Employee VALUES ('EM2', 'Fatimah', '02/06/1997', 'Flavor Development
Specialist', 'fatima@gmail.com', '647-333-5565', 50000.80, 'R2');
```

```

INSERT INTO Employee VALUES ('EM3', 'Bilal', '17/05/2000', 'Packaging Operator',
'bilal@gmail.com', '647-221-5499', 65000.80, 'R3');
INSERT INTO Employee VALUES ('EM4', 'Sayeed', '11/04/1995', 'Food Safety Operator',
'bilal@gmail.com', '647-321-8766', 90000.90, 'R4');
INSERT INTO Employee VALUES ('EM5', 'Alexis', '31/09/2000', 'Mainenance Technician',
'alexis@gmail', '647-441-4499', 80000.80, 'R5');

-- Order (Order_ID, Order_Employee_ID, Order_Icecream_ID, Order_Payment_ID,
Order_Status, Order_Total_Order, Order_Invoice_ID, Order_Date, Order_Customer_ID)
INSERT INTO Order_IceCream VALUES('OD1', 'EO1', 1, 'OP1', 'Order Complete', 160.50,
'OIN1', TO_DATE('04/12/2024', 'MM/DD/YYYY'), 'CC1');
INSERT INTO Order_IceCream VALUES('OD2', 'EO2', 2, 'OP2', 'Order Pending', 250.25,
'OIN2', TO_DATE('05/12/2023', 'MM/DD/YYYY'), 'CC2');
INSERT INTO Order_IceCream VALUES('OD3', 'EO3', 3, 'OP3', 'Order Shipping', 100.05,
'OIN3', TO_DATE('06/17/2024', 'MM/DD/YYYY'), 'CC3');
INSERT INTO Order_IceCream VALUES('OD4', 'EO4', 4, 'OP4', 'Order Completed', 170.30,
'OIN4', TO_DATE('01/04/2022', 'MM/DD/YYYY'), 'CC4');
INSERT INTO Order_IceCream VALUES('OD5', 'EO5', 5, 'OP5', 'Order Cancelled', 135.95,
'OIN5', TO_DATE('03/19/2019', 'MM/DD/YYYY'), 'CC5');

INSERT INTO PaymentInformation VALUES('401R', 'VISA', TO_DATE('02/03/2024',
'MM/DD/YYYY'), 'DONE', 2443.00, '4001', '120E');
INSERT INTO PaymentInformation VALUES('402R', 'MasterCard', TO_DATE('02/05/2024',
'MM/DD/YYYY'), 'PENDING', 1500.00, 'AA2', '121E');
INSERT INTO PaymentInformation VALUES('403R', 'PayPal', TO_DATE('02/10/2024',
'MM/DD/YYYY'), 'FAILED', 300.50, 'AA3', '122E');
INSERT INTO PaymentInformation VALUES('404R', 'American Express',
TO_DATE('02/12/2024', 'MM/DD/YYYY'), 'DONE', 750.75, 'AA4', '123E');
INSERT INTO PaymentInformation VALUES('405R', 'Debit Card', TO_DATE('02/15/2024',
'MM/DD/YYYY'), 'DONE', 120.00, 'AA5', '124E');

-- Invoice (Invoice_ID, Invoice_Order_ID, Invoice_Date, Invoice_Total_Amount,
Invoice_Status, Invoice_Customer_ID, Invoice_Payment_Information_ID)
-- Now you can insert into Invoice
INSERT INTO Invoice VALUES ('OIN1', 'OD1', '06/01/2024', 160.50, 'Paid', 'AA1', 'P1');
INSERT INTO Invoice VALUES ('OIN2', 'OD2', '07/01/2024', 250.25, 'Pending', 'AA2', 'P2');
INSERT INTO Invoice VALUES ('OIN3', 'OD3', '06/01/2024', 90.10, 'Paid', 'AA3', 'P3');

-- Address(CustomerID, AddressID, Country, Province, City, Location, PostalCode)

```

```
INSERT INTO Address VALUES('AA1', '4001', 'Canada', 'Ontario', 'Toronto', '341 Bloor St E',  
'L6V 4T3');  
INSERT INTO Address VALUES('AA2', '4002', 'Canada', 'Ontario', 'Toronto', '123 King St W',  
'M5H 1K4');  
INSERT INTO Address VALUES('AA3', '4003', 'Canada', 'Ontario', 'Mississauga', '456 Main St  
N', 'L5B 1M4');  
INSERT INTO Address VALUES('AA4', '4004', 'Canada', 'Ontario', 'Ottawa', '789 Elm St S',  
'K1A 0B1');  
INSERT INTO Address VALUES('AA5', '4005', 'Canada', 'Ontario', 'Hamilton', '321 Maple Ave',  
'L8P 1B5');  
  
-- has  
INSERT INTO has VALUES('AA1', 'OIN1');  
INSERT INTO has VALUES('AA2', 'OIN2');  
INSERT INTO has VALUES('AA3', 'OIN3');
```

## Populated Tables:

|   | ADDRESS_CUSTOMERID | ADDRESS_PROVINCE | ADDRESS_CITY | ADDRESS_LOCATION | ADDRESS_ID  | ADDRESS_COUNTRY | ADDRESS_POSTALCODE |
|---|--------------------|------------------|--------------|------------------|-------------|-----------------|--------------------|
| 1 | AA1                | 4001             | Canada       | Ontario          | Toronto     | 341 Bloor St E  | L6V 4T3            |
| 2 | AA2                | 4002             | Canada       | Ontario          | Toronto     | 123 King St W   | M5H 1K4            |
| 3 | AA3                | 4003             | Canada       | Ontario          | Mississauga | 456 Main St N   | L5B 1M4            |
| 4 | AA4                | 4004             | Canada       | Ontario          | Ottawa      | 789 Elm St S    | K1A 0B1            |
| 5 | AA5                | 4005             | Canada       | Ontario          | Hamilton    | 321 Maple Ave   | L8P 1B5            |

|   | CUSTOMER_ID | CUSTOMER_NAME | CUSTOMER_ADDRESS_ID                      | CUSTOMER_PHONE_NUMBER | CUSTOMER_EMAIL   | CUSTOMER_PAYMENT_ID |
|---|-------------|---------------|------------------------------------------|-----------------------|------------------|---------------------|
| 1 | AA1         | Azaan         | 341 Yonge St, Toronto, ON M4B7L          | 647-899-1234          | azaan@gmail.com  | P1                  |
| 2 | AA2         | Maryam        | 15 Wallingford Rd, North York, ON M3G2V1 | 647-343-1567          | maryam@gmail.com | P2                  |
| 3 | AA3         | Dan           | 27 Kings College Cir, Toronto, ON M5K1L1 | 647-256-1273          | dan@gmail.com    | P3                  |
| 4 | AA4         | Susan         | 300 Borough Dr, Scarborough, ON M1O4P5   | 647-999-1415          | susan@gmail.com  | P4                  |
| 5 | AA5         | Ali           | 245 Church St, Toronto, ON M1T6W1        | 647-766-1322          | ali@gmail.com    | P5                  |

|   | EMPLOYEE_ID | EMPLOYEE_NAME | EMPLOYEE_BIRTH_DATE | EMPLOYEE_ROLE                 | EMPLOYEE_EMAIL      | EMPLOYEE_PHONE_NUMBER | EMPLOYEE_SALARY | EMPLOYEE_ORDER_ID |
|---|-------------|---------------|---------------------|-------------------------------|---------------------|-----------------------|-----------------|-------------------|
| 1 | EM1         | Vaishnavi     | 02/02/1999          | Production Operator           | vaishnavi@gmail.com | 647-222-5555          | 60000.5         | R1                |
| 2 | EM2         | Fatimah       | 02/06/1997          | Flavor Development Specialist | fatima@gmail.com    | 647-333-5565          | 50000.8         | R2                |
| 3 | EM3         | Bilal         | 17/05/2000          | Packaging Operator            | bilal@gmail.com     | 647-221-5499          | 65000.8         | R3                |
| 4 | EM4         | Sayed         | 11/04/1995          | Food Safety Operator          | bilal@gmail.com     | 647-321-8766          | 90000.9         | R4                |
| 5 | EM5         | Alexis        | 31/09/2000          | Maintenance Technician        | alexis@gmail        | 647-441-4499          | 80000.8         | R5                |

|   | FLAVOUR_ID | FLAVOUR_TYPE        | FLAVOUR_ALLERGYINFO    | FLAVOUR_AVAILABLE | FLAVOUR_DESCRIPTION                                                 | FLAVOUR_INGREDIENTS            |
|---|------------|---------------------|------------------------|-------------------|---------------------------------------------------------------------|--------------------------------|
| 1 | 1          | Mint Chocolate Chip | No Nuts                | Y                 | Fresh minty ice cream with chocolate chips                          | mint, cocoa, ice cream         |
| 2 | 2          | Chocolate           | May contain Hazel nuts | Y                 | Rich chocolate ice cream made from high-quality cocoa               | cocoa, cream, sugar            |
| 3 | 3          | Vanilla             | No nuts                | N                 | Classic vanilla ice cream with a smooth texture                     | vanilla extract, cream, sugar  |
| 4 | 4          | Strawberry          | No nuts                | N                 | Fresh strawberry ice cream made with real strawberries              | strawberries, cream, sugar     |
| 5 | 5          | Cookie Dough        | May contain nuts       | Y                 | Delicious ice cream with chunks of cookie dough and chocolate chips | cookie dough, chocolate chips, |

|   | CUSTOMER_ID | INVOICE_ID |
|---|-------------|------------|
| 1 | AA1         | OIN1       |
| 2 | AA2         | OIN2       |
| 3 | AA3         | OIN3       |

|   | ICECREAM_ID | ICECREAM_FLAVOURID | ICECREAM_PRICEPERUNIT | ICECREAM_STOCKQUANTITY |
|---|-------------|--------------------|-----------------------|------------------------|
| 1 | 14          | 1                  | 2.29                  | 1000                   |
| 2 | 15          | 2                  | 2.49                  | 800                    |
| 3 | 16          | 3                  | 2.19                  | 500                    |
| 4 | 17          | 4                  | 2.39                  | 700                    |
| 5 | 18          | 5                  | 2.59                  | 600                    |

|   | INVOICE_ID | INVOICE_ORDER_ID | INVOICE_DATE | INVOICE_TOTAL_AMOUNT | INVOICE_STATUS | INVOICE_CUSTOMER_ID | INVOICE_PAYMENT_INFORMATION_ID |
|---|------------|------------------|--------------|----------------------|----------------|---------------------|--------------------------------|
| 1 | OIN1       | OD1              | 06/01/2024   | 160.5                | Paid           | AA1                 | P1                             |
| 2 | OIN2       | OD2              | 07/01/2024   | 250.25               | Pending        | AA2                 | P2                             |
| 3 | OIN3       | OD3              | 06/01/2024   | 90.1                 | Paid           | AA3                 | P3                             |

|   | ORDER_ID | ORDER_EMPLOYEE_ID | ORDER_ICECREAM_ID | ORDER_PAYMENT_ID | ORDER_ORDER_STATUS | ORDER_TOTAL_ORDER | ORDER_INVOICE_ID | ORDER_DATE | ORDER_CUSTOMER_ID |
|---|----------|-------------------|-------------------|------------------|--------------------|-------------------|------------------|------------|-------------------|
| 1 | OD1      | E01               | 1                 | OP1              | Order Complete     | 160.5             | OIN1             | 12-APR-24  | CC1               |
| 2 | OD2      | E02               | 2                 | OP2              | Order Pending      | 250.25            | OIN2             | 12-MAY-23  | CC2               |
| 3 | OD3      | E03               | 3                 | OP3              | Order Shipping     | 100.05            | OIN3             | 17-JUN-24  | CC3               |
| 4 | OD4      | E04               | 4                 | OP4              | Order Completed    | 170.3             | OIN4             | 04-JAN-22  | CC4               |
| 5 | OD5      | E05               | 5                 | OP5              | Order Cancelled    | 135.95            | OIN5             | 19-MAR-19  | CC4               |

|   | ⚡ PAYMENT_ID | ⚡ METHOD_OF_PAYMENT | ⚡ PAYMENT_DATE | ⚡ PAYMENT_STATUS | ⚡ PAYMENT_AMOUNT | ⚡ CUSTOMER_ID | ⚡ INVOICE_ID |
|---|--------------|---------------------|----------------|------------------|------------------|---------------|--------------|
| 1 | 401R         | VISA                | 03-FEB-24      | DONE             | 2443             | 4001          | 120E         |
| 2 | 402R         | MasterCard          | 05-FEB-24      | PENDING          | 1500             | AA2           | 121E         |
| 3 | 403R         | PayPal              | 10-FEB-24      | FAILED           | 300.5            | AA3           | 122E         |
| 4 | 404R         | American Express    | 12-FEB-24      | DONE             | 750.75           | AA4           | 123E         |
| 5 | 405R         | Debit Card          | 15-FEB-24      | DONE             | 120              | AA5           | 124E         |

## Assignment 4a: Simple Queries

### Simple Query 1:

**Explanation:** Displays all ice cream flavours that have no nuts.

```
SELECT DISTINCT
  Flavour_ID as "Flavour_ID",
  Flavour_Type as "Flavour Type"
FROM Flavour
WHERE Flavour_AllergyInfo = 'No nuts'
```

|   | ⚡ Flavour_ID | ⚡ Flavour Type      |
|---|--------------|---------------------|
| 1 | 1            | Mint Chocolate Chip |
| 2 | 3            | Vanilla             |
| 3 | 4            | Strawberry          |

### Simple Query 2:

**Explanation:** Displays the data of the staff from Employee ID.

```
SELECT
  Employee_ID as "Employee ID",
  Employee_Name as "Employee Name",
  Employee_Birth_Date as "Birth Date",
  Employee_Role as "Role",
  Employee_Email as "Email",
  Employee_Phone_Number as "Phone Number",
  Employee_Salary as "Salary",
  Employee_Order_ID as "Order_ID"
FROM Employee
```



ORDER BY Employee\_ID;

| Employee ID | Employee Name | Birth Date | Role                          | Email               | Phone Number | Salary  | Order_ID |
|-------------|---------------|------------|-------------------------------|---------------------|--------------|---------|----------|
| 1 EM1       | Vaishnavi     | 02/02/1999 | Production Operator           | vaishnavi@gmail.com | 647-222-5555 | 60000.5 | OD1      |
| 2 EM2       | Fatimah       | 02/06/1997 | Flavor Development Specialist | fatima@gmail.com    | 647-333-5565 | 50000.8 | OD2      |
| 3 EM3       | Bilal         | 17/05/2000 | Packaging Operator            | bilal@gmail.com     | 647-221-5499 | 65000.8 | R3       |
| 4 EM4       | Sayeed        | 11/04/1995 | Food Safety Operator          | bilal@gmail.com     | 647-321-8766 | 90000.9 | R4       |
| 5 EM5       | Alexis        | 31/09/2000 | Maintenance Technician        | alexis@gmail        | 647-441-4499 | 80000.8 | R5       |

### Simple Query 3:

**Explanation:** Displays all the number of ice creams that are greater than or equal to 700.

```
SELECT *
FROM IceCream
WHERE IceCream_StockQuantity >= 700;
```

|   | ICECREAM_ID | ICECREAM_FLAVOURID | ICECREAM_PRICEPERUNIT | ICECREAM_STOCKQUANTITY |
|---|-------------|--------------------|-----------------------|------------------------|
| 1 | 14          | 1                  | 2.29                  | 1000                   |
| 2 | 15          | 2                  | 2.49                  | 800                    |
| 3 | 17          | 4                  | 2.39                  | 700                    |

### Simple Query 4:

**Explanation:** Displays all customers with active orders.

```
SELECT o.Order_ID,o.Order_Total_Order
FROM Order_IceCream o
WHERE o.Order_Order_Status IN ('Order Complete', 'Order Pending', 'Order Shipping')
ORDER BY o.Order_ID ASC; --will show the orders in Order ID Ascending order
```

|   | ORDER_ID | ORDER_TOTAL_ORDER |
|---|----------|-------------------|
| 1 | OD1      | 160.5             |
| 2 | OD2      | 250.25            |
| 3 | OD3      | 100.05            |

### Simple Query 5:

**Explanation:** Display all invoices along with payment status.

```
SELECT DISTINCT
    Invoice_ID as "Invoice ID",
    Invoice_Total_Amount as "Total Amount",
    Invoice_Status as "Payment Status"
FROM Invoice
ORDER BY Invoice_ID;
```

|   | Invoice ID | Total Amount | Payment Status |
|---|------------|--------------|----------------|
| 1 | OIN1       | 160.5        | Paid           |
| 2 | OIN2       | 250.25       | Pending        |
| 3 | OIN3       | 90.1         | Paid           |

### Simple Query 6:

**Explanation:** Displays all employees who are a Production Operator.

```
SELECT
    Employee_ID as "Employee ID",
    Employee_Name as "Employee Name",
    Employee_Salary as "Salary"
FROM Employee
WHERE Employee_Role = 'Production Operator'
ORDER BY Employee_Name;
```

|   | Employee ID | Employee Name | Salary  |
|---|-------------|---------------|---------|
| 1 | EM1         | Vaishnavi     | 60000.5 |

### Simple Query 7:

**Explanation:** Stores information about the customer's address. These customers include well-known stores such as Walmart, Shoppers Drug Mart, Frescho, Costco, and Nofrills.

```
SELECT
    Address_CustomerID as "Customer ID",
    Address_ID as "Address ID",
```

Address\_Country as "Country",  
 Address\_Location as "Location",  
 Address\_Province as "Province",  
 Address\_City as "City",  
 Address\_PostalCode as "Postal Code"

FROM Address  
 ORDER BY Address\_ID;

|   | Customer ID | Address ID | Country        | Location    | Province | City    | Postal Code |
|---|-------------|------------|----------------|-------------|----------|---------|-------------|
| 1 | C1          | 4001       | 341 Bloor St E | Toronto     | Canada   | Ontario | L6V 4T3     |
| 2 | C2          | 4002       | 123 King St W  | Toronto     | Canada   | Ontario | M5H 1K4     |
| 3 | C3          | 4003       | 456 Main St N  | Mississauga | Canada   | Ontario | L5B 1M4     |
| 4 | C4          | 4004       | 789 Elm St S   | Ottawa      | Canada   | Ontario | K1A 0B1     |
| 5 | C5          | 4005       | 321 Maple Ave  | Hamilton    | Canada   | Ontario | L8P 1B5     |

### Simple Query 8:

**Explanation:** Displays the Customer ID that paid with VISA.

SELECT  
 Customer\_ID as "Customer ID",  
 Invoice\_ID as "Invoice ID",  
 Payment\_ID as "Payment ID",  
 Method\_of\_Payment as "Payment Method",  
 Payment\_Date as "Payment Date",  
 Payment\_Amount as "Payment Amount",  
 Payment\_Status as "Payment Status"  
 FROM PaymentInformation  
 WHERE Method\_of\_Payment = 'VISA'  
 ORDER BY Payment\_Date

|   | Customer ID | Invoice ID | Payment ID | Payment Method | Payment Date | Payment Amount | Payment Status |
|---|-------------|------------|------------|----------------|--------------|----------------|----------------|
| 1 | 4001        | 120E       | 401R       | VISA           | 24-02-03     | 2443           | DONE           |

### Simple Query 9:

**Explanation:** Provides information about the number of employees there are.

```
SELECT
    Employee_Role as "Role",
    COUNT(Employee_ID) as " Total Number of Employees"
FROM Employee
GROUP BY Employee_Role
```

|   | Role                          | Total Number of Employees |
|---|-------------------------------|---------------------------|
| 1 | Production Operator           | 1                         |
| 2 | Flavor Development Specialist | 1                         |
| 3 | Packaging Operator            | 1                         |
| 4 | Mainetance Technician         | 1                         |
| 5 | Food Safety Operator          | 1                         |

### Assignment 4b: Designing Views/Advanced Queries

#### Advanced Query 1:

**Explanation:** All flavours that cost more than the average price of all the ice creams.

```
--All flavours that cost more than the average price of all the ice creams
SELECT f.Flavour_Type, ic.Icecream_PricePerUnit --ic is aliased in the next line
FROM Flavour f
JOIN IceCream ic ON f.Flavour_ID = ic.Icecream_FlavourID
WHERE ic.Icecream_PricePerUnit > (
    SELECT AVG(Icecream_PricePerUnit)
    FROM IceCream
);
```

|   | FLAVOUR_TYPE | ICECREAM_PRICEPERUNIT |
|---|--------------|-----------------------|
| 1 | Chocolate    | 2.49                  |
| 2 | Cookie Dough | 2.59                  |

## Advanced Query 2:

**Explanation:** Top 3 flavours that contain the MOST amount of ingredients.

--Top 3 flavours that contain the MOST amount of ingredients

```
SELECT *
FROM (
    SELECT f.Flavour_Type,
           LENGTH(f.Flavour_Ingredients) - LENGTH(REPLACE(f.Flavour_Ingredients, ',', '')) + 1
    AS ingredientCount
    FROM Flavour f
    ORDER BY ingredientCount DESC
)
WHERE ROWNUM <= 3; --CHANGE THE 3 HERE TO GET MORE VALUES
```

|   | FLAVOUR_TYPE        | INGREDIENTCOUNT |
|---|---------------------|-----------------|
| 1 | Mint Chocolate Chip | 3               |
| 2 | Chocolate           | 3               |
| 3 | Vanilla             | 3               |

## Advanced Query 3:

**Explanation:** Top 3 ice creams with the highest amount of in stock quantity.

```
SELECT *
FROM (
    SELECT ic.Icecream_ID, f.Flavour_Type, ic.Icecream_StockQuantity
    FROM IceCream ic
    JOIN Flavour f ON ic.Icecream_FlavourID = f.Flavour_ID
    ORDER BY ic.Icecream_StockQuantity DESC
)
WHERE ROWNUM <= 3;
```

|   | ICECREAM_ID | FLAVOUR_TYPE        | ICECREAM_STOCKQUANTITY |
|---|-------------|---------------------|------------------------|
| 1 | 14          | Mint Chocolate Chip | 1000                   |
| 2 | 15          | Chocolate           | 800                    |
| 3 | 17          | Strawberry          | 700                    |

#### Advanced Query 4:

**Explanation:** The price of each ice cream, and how much more or less it is than the average.

```
SELECT f.Flavour_Type, ic.Icecream_PricePerUnit,
       (ic.Icecream_PricePerUnit - (SELECT AVG(Icecream_PricePerUnit) FROM IceCream)) AS
priceDifference --we can simply use the AVG function here to get the averages
FROM IceCream ic
JOIN Flavour f ON ic.Icecream_FlavourID = f.Flavour_ID;
```

|   | FLAVOUR_TYPE        | ICECREAM_PRICEPERUNIT | PRICEDIFFERENCE |
|---|---------------------|-----------------------|-----------------|
| 1 | Mint Chocolate Chip | 2.29                  | -0.1            |
| 2 | Chocolate           | 2.49                  | 0.1             |
| 3 | Vanilla             | 2.19                  | -0.2            |
| 4 | Strawberry          | 2.39                  | 0               |
| 5 | Cookie Dough        | 2.59                  | 0.2             |

#### Advanced Query 5:

**Explanation:** The amount of salary is earned by each role.

```
-- how much salary is earned by each role?
SELECT e.Employee_Role, SUM(e.Employee_Salary) AS totalSalary
FROM Employee e
GROUP BY e.Employee_Role
ORDER BY totalSalary ASC;
```

|   | EMPLOYEE_ROLE                 | TOTALSALARY |
|---|-------------------------------|-------------|
| 1 | Flavor Development Specialist | 50000.8     |
| 2 | Production Operator           | 60000.5     |
| 3 | Packaging Operator            | 65000.8     |
| 4 | Maintenance Technician        | 80000.8     |
| 5 | Food Safety Operator          | 90000.9     |

### Advanced Query 6:

**Explanation:** Displays the employees that have not been assigned any orders.

--which employees are slacking? i.e which employees have not been assigned any orders?

```
SELECT e.Employee_Name
FROM Employee e
WHERE NOT EXISTS (
    SELECT 1
    FROM Order_IceCream oi
    WHERE oi.Order_Employee_ID = e.Employee_ID
);
```

|   | EMPLOYEE_NAME |
|---|---------------|
| 1 | Vaishnavi     |
| 2 | Fatimah       |
| 3 | Bilal         |
| 4 | Sayeed        |
| 5 | Alexis        |

### Advanced Query 7:

**Explanation:** TOTAL revenue generated by ALL payment methods.

--TOTAL revenue generated by ALL payment method

```
SELECT pi.Method_Of_Payment, SUM(pi.Payment_Amount) AS totalRevenue
```

```

FROM PaymentInformation pi
GROUP BY pi.Method_Of_Payment
ORDER BY totalRevenue ASC;

```

|   | METHOD_OF_PAYMENT | TOTALREVENUE |
|---|-------------------|--------------|
| 1 | Debit Card        | 120          |
| 2 | PayPal            | 300.5        |
| 3 | American Express  | 750.75       |
| 4 | MasterCard        | 1500         |
| 5 | VISA              | 2443         |

### Advanced Query 8:

**Explanation:** Customers with a relatively similar postal code, this is to determine which customers are located closer to each other.

--CUSTOMERS with a relatively similiar postal code, this is to determine which customers are located closer to each other

```

SELECT SUBSTR(a.Address_PostalCode, 1, 3) AS postalArea, c.Customer_Name
FROM Address a
JOIN Customer c ON a.Address_CustomerID = c.Customer_ID
WHERE ROWNUM <= 3
ORDER BY SUBSTR(a.Address_PostalCode, 1, 3), c.Customer_Name;

```

|   | POSTALAREA | CUSTOMER_NAME      |
|---|------------|--------------------|
| 1 | L5B        | Costco             |
| 2 | L6V        | Walmart            |
| 3 | M5H        | Shoppers Drug Mart |



### Advanced Query 9:

**Explanation:** Counts the number of customers per city and show their names separated by commas.

```
--COUNT the number of customers per city and show their names separated by commas
SELECT a.Address_City, COUNT(*) AS customerCount, LISTAGG(c.Customer_Name, ', ')
WITHIN GROUP (ORDER BY c.Customer_Name) AS customerNames
FROM Address a
JOIN Customer c ON a.Address_CustomerID = c.Customer_ID
GROUP BY a.Address_City
ORDER BY customerCount DESC;
```

|   | ADDRESS_CITY | CUSTOMERCOUNT | CUSTOMERNAMES                                          |
|---|--------------|---------------|--------------------------------------------------------|
| 1 | Ontario      | 5             | Costco, Freshco, Nofrills, Shoppers Drug Mart, Walmart |

### Advanced Query 10:

**Explanation:** Finding ALL orders that cost more than the average of all the orders.

```
--Finding ALL orders that cost more than the average of all the orders
SELECT oi.Order_ID, oi.Order_Total_Order
FROM Order_IceCream oi
WHERE oi.Order_Total_Order > (
    SELECT AVG(oi2.Order_Total_Order)
    FROM Order_IceCream oi2
)
ORDER BY oi.Order_Total_Order DESC;
```

|   | ORDER_ID | ORDER_TOTAL_ORDER |
|---|----------|-------------------|
| 1 | OD2      | 250.25            |
| 2 | OD4      | 170.3             |

### Advanced Query 11:

**Explanation:** Top three oldest UNPAID invoices, this will include the canceled invoices too.

--Top 3 oldest UNPAID invoices, this will include the canceled invoices too

```
SELECT *  
FROM (  
    SELECT i.Invoice_ID, i.Invoice_Date, i.Invoice_Status  
    FROM Invoice i  
    WHERE i.Invoice_Status != 'Paid'  
    ORDER BY TO_DATE(i.Invoice_Date, 'DD/MM/YYYY') ASC  
)  
WHERE ROWNUM <= 3;
```

|   | INVOICE_ID | INVOICE_DATE | INVOICE_STATUS |
|---|------------|--------------|----------------|
| 1 | OIN2       | 24-07-01     | Pending        |

### Advanced Query 12:

**Explanation:** Total paid invoices for each month.

--Total paid invoices for each month

```
SELECT EXTRACT(MONTH FROM TO_DATE(i.Invoice_Date, 'DD/MM/YYYY')) AS  
invoiceMonth, SUM(i.Invoice_Total_Amount) AS totalPaid  
FROM Invoice i  
WHERE i.Invoice_Status = 'Paid'  
GROUP BY EXTRACT(MONTH FROM TO_DATE(i.Invoice_Date, 'DD/MM/YYYY'))  
ORDER BY invoiceMonth;
```

|   | INVOICEMONTH | TOTALPAID |
|---|--------------|-----------|
| 1 | 6            | 250.6     |

### View 1:

**Explanation:** Displays employees that have a high salary (salary > 70000).

```
--Displays employees that have a high salary (salary > 70000)
CREATE VIEW Employee_High_Salary AS
SELECT Employee_Name, Employee_Salary
FROM Employee
WHERE Employee_Salary > 70000;
```

|   | EMPLOYEE_NAME | EMPLOYEE_SALARY |
|---|---------------|-----------------|
| 1 | Sayed         | 90000.9         |
| 2 | Alexis        | 80000.8         |

## View 2:

**Explanation:** Displays the customers whose order is complete.

```
--Displays the customers whose order is complete
CREATE VIEW Orders_Complete AS
SELECT Order_ID, Order_Order_Status
FROM Order_IceCream
WHERE Order_Order_Status = 'Order Complete';
```

|   | ORDER_ID | ORDER_ORDER_STATUS |
|---|----------|--------------------|
| 1 | OD1      | Order Complete     |
| 2 | OD4      | Order Complete     |

## View 3:

**Explanation:** Display the top 3 oldest unpaid invoices.

```
--Display table: display the top 3 oldest unpaid invoices.
CREATE VIEW Unpaid_Invoices AS
SELECT Invoice_ID, Invoice_Date, Invoice_Status
FROM Invoice
WHERE Invoice_Status != 'Paid'
ORDER BY TO_DATE(Invoice_Date, 'DD/MM/YYYY') ASC;
```

|   | INVOICE_ID | INVOICE_DATE | INVOICE_STATUS |
|---|------------|--------------|----------------|
| 1 | OIN2       | 24-07-01     | Pending        |

#### View 4:

**Explanation:** Ontario Based Addresses in which the order is cancelled.

--Display table: Ontario based addresses

CREATE VIEW Canceled\_Orders AS

SELECT

Order\_ID,  
Order\_Employee\_ID,  
Order\_Icecream\_ID,  
Order\_Payment\_ID,  
Order\_Total\_Order,  
Order\_Invoice\_ID,  
Order\_Date,  
Order\_Customer\_ID

FROM

Order\_IceCream

WHERE

Order\_Order\_Status = 'Order Cancelled';

| ORDER_ID | ORDER_EMPLOYEE_ID | ORDER_ICECREAM_ID | ORDER_PAYMENT_ID | ORDER_TOTAL_ORDER | ORDER_INVOICE_ID | ORDER_DATE | ORDER_CUSTOMER_ID |
|----------|-------------------|-------------------|------------------|-------------------|------------------|------------|-------------------|
| 1 ODS    | E05               | 5 OF5             |                  | 135.95 OIN5       |                  | 19-03-19   | CC5               |

### Assignment 5: Interesting Queries and UnixShell

#### Interesting Query 1:

**Explanation:** Displays all the customers and flavours

--Interesting Query 1: UNION

```

SELECT
    Flavour_Type AS Name
FROM
    Flavour
UNION
SELECT
    Customer_Name AS Name
FROM
    Customer;

```

|    | NAME                |
|----|---------------------|
| 1  | Chocolate           |
| 2  | Cookie Dough        |
| 3  | Costco              |
| 4  | Freshco             |
| 5  | Mint Chocolate Chip |
| 6  | Nofrills            |
| 7  | Shoppers Drug Mart  |
| 8  | Strawberry          |
| 9  | Vanilla             |
| 10 | Walmart             |

## Interesting Query 2:

**Explanation:** Displays flavours that might have Hazel nuts.

```

-- Interesting Query 2: EXISTS
-- Displays flavours that might have Hazel nuts.
SELECT DISTINCT
    f.Flavour_ID AS "Flavour_ID",
    f.Flavour_Type AS "Flavour Type"
FROM Flavour f
WHERE EXISTS (
    SELECT 1
    FROM Flavour f2
    WHERE f2.Flavour_ID = f.Flavour_ID
    AND f2.Flavour_AllergyInfo = 'May contain Hazel nuts'
)

```

);

|   | Flavour_ID | Flavour Type |
|---|------------|--------------|
| 1 | 2          | Chocolate    |

### Interesting Query 3:

**Explanation:** Displays the invoices of every payment status and orders the results through descending order.

```
SELECT
    Invoice_Status as "Payment Status",
    COUNT(Invoice_ID) as "Number of Invoices"
FROM Invoice
GROUP BY Invoice_Status
ORDER BY "Number of Invoices" DESC;
```

|   | Payment Status | Number of Invoices |
|---|----------------|--------------------|
| 1 | Paid           | 2                  |
| 2 | Pending        | 1                  |

### Interesting Query 4:

**Explanation:** Displays the invoices based on payment status except for Paid

```
-- Interesting Query 4: MINUS
-- Displays the invoices based on payment status except for Paid
SELECT
    Invoice_Status as "Payment Status",
    COUNT(Invoice_ID) as "Number of Invoices"
FROM Invoice
GROUP BY Invoice_Status
MINUS
SELECT
    Invoice_Status as "Payment Status",
    COUNT(Invoice_ID) as "Number of Invoices"
```

```

FROM Invoice
WHERE Invoice_Status = 'Paid'
GROUP BY Invoice_Status
ORDER BY "Number of Invoices" DESC;

```

|   | Payment Status | Number of Invoices |
|---|----------------|--------------------|
| 1 | Pending        | 1                  |

### Interesting Query 5:

**Explanation:** Displays The employees who do not have any orders assigned

```

-- Interesting Query 5: HAVING
-- Displays The employees who do not have any orders assigned
SELECT
    e.Employee_Name
FROM
    Employee e
LEFT JOIN
    Order_IceCream oi ON e.Employee_ID = oi.Order_Employee_ID
GROUP BY
    e.Employee_Name
HAVING
    COUNT(oi.Order_ID) = 0;

```

|   | EMPLOYEE_NAME |
|---|---------------|
| 1 | Fatimah       |
| 2 | Bilal         |
| 3 | Vaishnavi     |
| 4 | Sayeed        |
| 5 | Alexis        |

## Unix Shell Menu Implementation:

### Menu Screen:

```
=====
| Oracle All Inclusive Tool                               |
| Main Menu - Select Desired Operation(s):                |
| <CTRL-Z Anytime to Enter Interactive CMD Prompt>        |
|-----|
M) View Manual
1) Drop Tables
2) Create Tables
3) Populate Tables
4) Query Tables
E) End/Exit
Choose:
█
```



## Drop Tables:

```
1
SQL*Plus: Release 12.1.0.2.0 Production on Thu Oct 24 01:48:09 2024
Copyright (c) 1982, 2014, Oracle. All rights reserved.

Connected to:
Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options

SQL> SQL> SQL> SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL>
Table dropped.

SQL> SQL> Disconnected from Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options
Press [Enter] to continue...
```

## Create Tables:

```
2
SQL*Plus: Release 12.1.0.2.0 Production on Thu Oct 24 01:49:16 2024
Copyright (c) 1982, 2014, Oracle. All rights reserved.

Connected to:
Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options

SQL> SQL> SQL> 2 3 4 5 6 7 8
Table created.

SQL> SQL> 2 3 4 5 6 7
Table created.

SQL> SQL> 2 3 4 5 6 7 8
Table created.

SQL> SQL> 2 3 4 5 6 7 8 9 10
Table created.

SQL> SQL> 2 3 4 5 6 7 8 9 10 11
Table created.

SQL> SQL> 2 3 4 5 6 7 8 9 10
Table created.

SQL> SQL> 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Table created.

SQL> SQL> 2 3 4 5 6 7 8 9 10 11 12 13
Table created.

SQL> SQL> 2 3 4 5 6 7
Table created.

SQL> SQL> Disconnected from Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options
Press [Enter] to continue...
```

## Populate Tables:

```
Choose:
3
populate_tables.sh: line 56: warning: here-document at line 2 delimited by end-of-file (wanted `EOF')

SQL*Plus: Release 12.1.0.2.0 Production on Thu Oct 24 01:50:36 2024

Copyright (c) 1982, 2014, Oracle. All rights reserved.

Connected to:
Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options

SQL> SQL> SQL> SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL> SQL> SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.
```

```
SQL> SQL> SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL> SQL> SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL> SQL> SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL>  
1 row created.
```

```
SQL>
1 row created.

SQL> SQL> SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL> SQL> SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL> SQL> SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL>
1 row created.

SQL> Disconnected from Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options
```

## Query Tables:

```
4

SQL*Plus: Release 12.1.0.2.0 Production on Thu Oct 24 01:51:53 2024

Copyright (c) 1982, 2014, Oracle. All rights reserved.

Connected to:
Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options

SQL> SQL> SQL> SQL> 2 3 4 5
Flavour_ID Flavour Type
-----
1 Mint Chocolate Chip
3 Vanilla
4 Strawberry

SQL> SQL> SQL> 2 3 4 5 6 7 8 9 10
Employee ID Employee Name
-----
Birth Date Role
-----
Email Phone Number Salary
-----
EM1 Vaishnavi
02-FEB-99 Production Operator
vaishnavi@gmail.com 647-222-5555 60000.5

EM2 Fatimah
02-JUN-97 Flavor Development Specialist
fatima@gmail.com 647-333-5565 50000.8

Employee ID Employee Name
-----
Birth Date Role
-----
Email Phone Number Salary
-----
EM3 Bilal
```

```

EM3          Bilal
17-MAY-00    Packaging Operator
bilal@gmail.com      647-221-5499      65000.8

```

```

EM4          Sayeed
11-APR-95    Food Safety Operator

```

```

Employee ID      Employee Name
-----
Birth Date      Role
-----
Email            Phone Number      Salary
-----
sayeed@gmail.com      647-321-8766      90000.9

```

```

EM5          Alexis
30-SEP-00    Maintenance Technician
alexis@gmail.com      647-441-4499      80000.8

```

```

SQL> SQL> SQL> 2 3
ICECREAM_ID ICECREAM_FLAVOURID ICECREAM_PRICEPERUNIT ICECREAM_STOCKQUANTITY
-----
14          1          2.29          1000
15          2          2.49          800
17          4          2.39          700

```

```

SQL> SQL> SQL> 2 3 4
ORDER_ID      ORDER_TOTAL_ORDER
-----
OD1          160.5
OD2          250.25
OD3          100.05
OD4          170.3

```

```

SQL> SQL> SQL> SQL> 2 3 4 5 6
Invoice ID      Total Amount Payment Status
-----
OIN1          160.5 Paid
OIN2          250.25 Pending
OIN3          90.1 Paid

```

```

SQL> SQL> SQL> SQL> 2 3 4 5 6 7
Employee ID      Employee Name
-----
Salary
-----
EM1          Vaishnavi
60000.5

```

```

SQL> SQL> SQL> SQL> 2 3 4 5 6 7 8 9 10 11 12 SQL> SQL> SQL> 2 3 4 5 6 SQL> SQL> SQL> SQL> SQL> 2 3 4 5
6 7
FLAVOUR_TYPE      ICECREAM_PRICEPERUNIT
-----
Chocolate          2.49
Cookie Dough      2.59

SQL> SQL> SQL> 2 3 4 5 6 7 8 9 SQL> SQL> SQL> 2 3 4 5 6 7 8
ICECREAM_ID FLAVOUR_TYPE
-----
14 Mint Chocolate Chip
1000
15 Chocolate
800
17 Strawberry
700

```

```

SQL> SQL> SQL> SQL> 2 3 4 5 6 7 8
ICECREAM_ID FLAVOUR_TYPE
-----
ICECREAM_STOCKQUANTITY
-----
14 Mint Chocolate Chip
1000

15 Chocolate
800

17 Strawberry
700

SQL> SQL> SQL> 2 3 4
FLAVOUR_TYPE ICECREAM_PRICEPERUNIT
-----
PRICEDIFFERENCE
-----
Mint Chocolate Chip 2.29
-.1

Chocolate 2.49
.1

Vanilla 2.19
-.2

FLAVOUR_TYPE ICECREAM_PRICEPERUNIT
-----
PRICEDIFFERENCE
-----
Strawberry 2.39
0

Cookie Dough 2.59
.2

```



```
SQL> SQL> SQL> SQL> 2 3 4
EMPLOYEE_ROLE TOTALSALARY
-----
Flavor Development Specialist 50000.8
Production Operator 60000.5
Packaging Operator 65000.8
Maintenance Technician 80000.8
Food Safety Operator 90000.9
```

```
SQL> SQL> SQL> 2 3 4 5 6 7
no rows selected
```

```
SQL> SQL> SQL> 2 3 4
METHOD_OF_PAYMENT TOTALREVENUE
-----
Debit Card 120
PayPal 300.5
American Express 750.75
MasterCard 1500
VISA 2443
```

```
SQL> SQL> SQL> SQL> 2 3 4 5
POSTALAREA CUSTOMER_NAME
-----
L5B Costco
L6V Walmart
M5H Shoppers Drug Mart
```

```
SQL> SQL> SQL> 2 3 4 5
ADDRESS_CITY
```

```
-----
CUSTOMERCOUNT
```

```
-----
CUSTOMERNAMES
```

```
-----
Canada
```

```
4
```

```
Costco, Freshco, Shoppers Drug Mart, Walmart
```

```
SQL> SQL> SQL> SQL> 2 3 4 5 6 7
ORDER_ID ORDER_TOTAL_ORDER
-----
OD2 250.25
OD4 170.3
```

```
SQL> SQL> SQL> SQL> 2 3 4 5 6 7 8
INVOICE_ID INVOICE_DATE
-----
INVOICE_STATUS
-----
OIN2 01-JUL-24
Pending
```

```
SQL> SQL> SQL> 2 3 4 5
INVOICEMONTH TOTALPAID
-----
6 250.6
```

```
SQL> SQL> SQL> SQL> SQL> SQL> SQL> SQL> SQL> SQL> SQL> SQL> SQL> SQL> SQL> SQL> 2 3 4 5 6 7 8 9
NAME
-----
Chocolate
Cookie Dough
Costco
Freshco
Mint Chocolate Chip
Nofrills
Shoppers Drug Mart
Strawberry
Vanilla
Walmart
```

```
10 rows selected.
```

## Assignment 6: Normalization/Functional Dependencies

### **Flavour:**

{Flavour\_ID} → Flavour\_Type, Flavour\_AllergyInfo, Flavour\_Available, Flavour\_Description, Flavour\_Ingredients

### **Customer:**

{Customer\_ID} → Customer\_Name, Customer\_Address\_ID, Customer\_Phone\_Number, Customer\_Email, Customer\_Payment\_ID

### **Employee:**

{Employee\_ID} → Employee\_Name, Employee\_Birth\_Date, Employee\_Role, Employee\_Email, Employee\_Phone\_Number, Employee\_Salary, Employee\_Order\_ID

### **IceCream:**

{Icecream\_ID} → Icecream\_FlavourID, Icecream\_PricePerUnit, Icecream\_StockQuantity

### **Order\_IceCream:**

{Order\_ID} → Order\_Employee\_ID, Order\_IceCream\_ID, Order\_Payment\_ID, Order\_Order\_Status, Order\_Total\_Order, Order\_Invoice\_ID, Order\_Date, Order\_Customer\_ID

### **Address:**

{Address\_CustomerID, Address\_ID} → Address\_Province, Address\_City, Address\_Location, Address\_Country, Address\_PostalCode

### **PaymentInformation:**

{Payment\_ID} → Method\_of\_Payment, Payment\_Date, Payment\_Status, Payment\_Amount, Customer\_ID, Invoice\_ID

### **Invoice:**

{Invoice\_ID} → Invoice\_Order\_ID, Invoice\_Date, Invoice\_Total\_Amount, Invoice\_Status, Invoice\_Customer\_ID, Invoice\_Payment\_Information\_ID

### **has:**

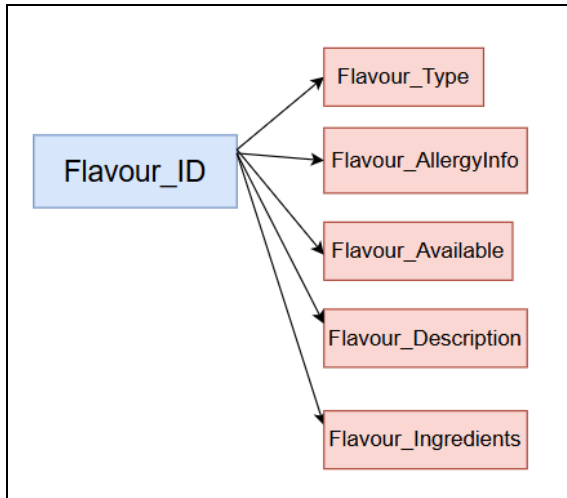
{Customer\_ID, Invoice\_ID} → Customer\_ID

{Customer\_ID, Invoice\_ID} → Invoice\_ID

## Assignment 7: Normalization/3NF

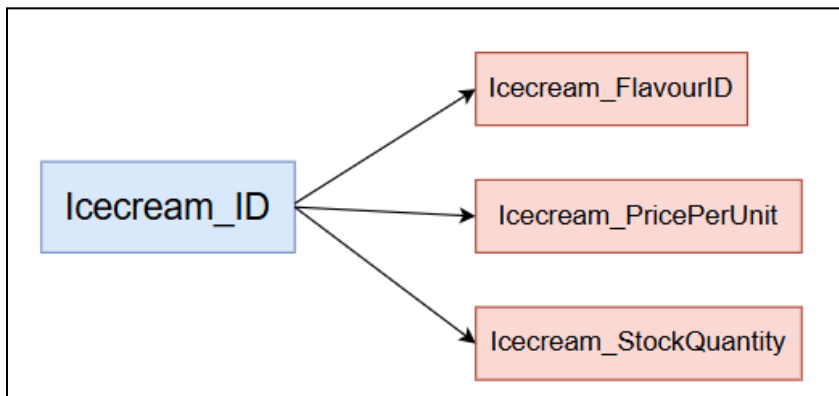
### Flavour:

{Flavour\_ID} → Flavour\_Type, Flavour\_AllergyInfo, Flavour\_Available, Flavour\_Description, Flavour\_Ingredients



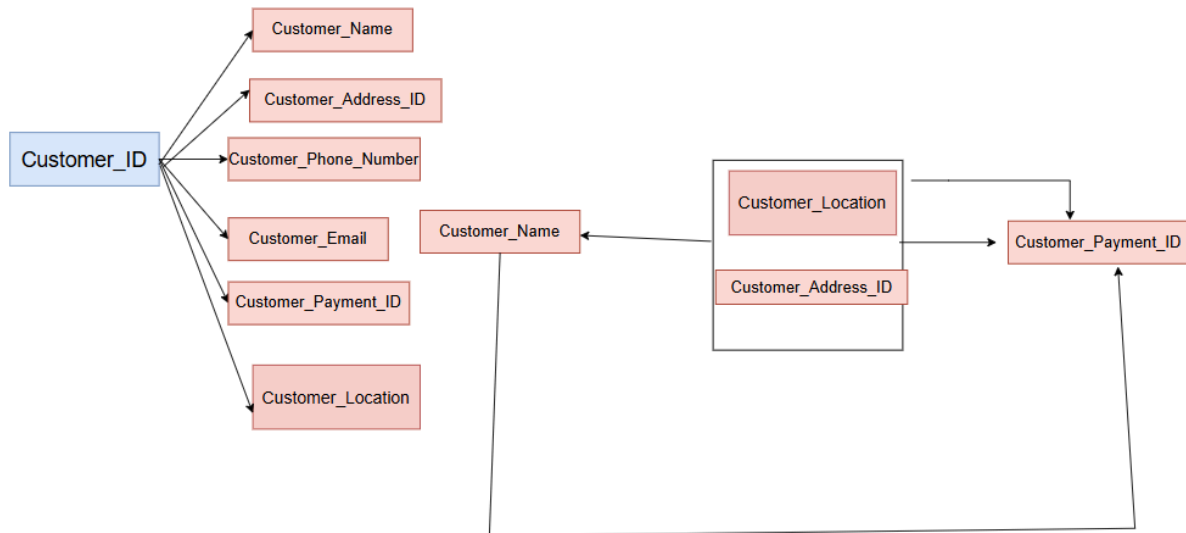
### IceCream:

{Icecream\_ID} → Icecream\_FlavourID, Icecream\_PricePerUnit, Icecream\_StockQuantity

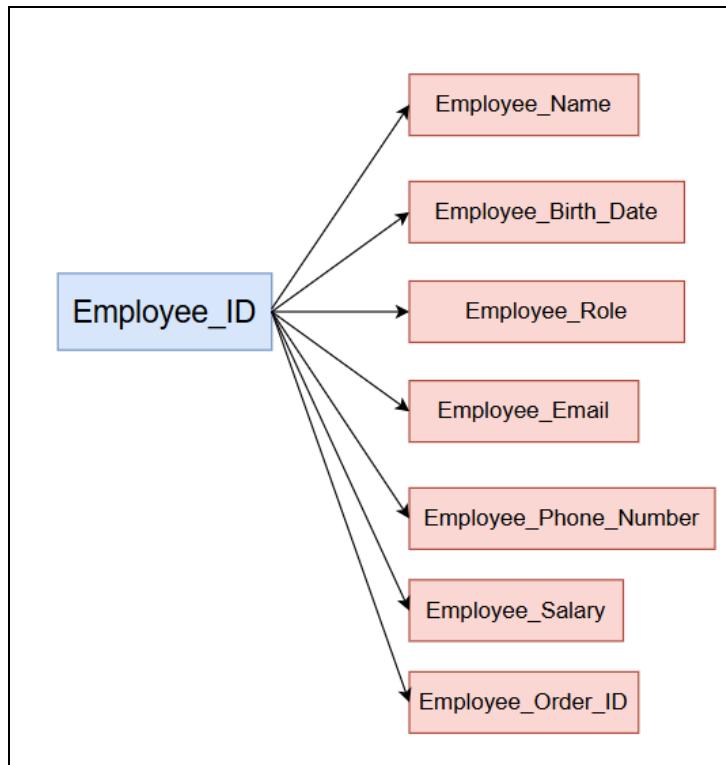


**Customer:**

{Customer\_ID} → Customer\_Name, Customer\_Address\_ID, Customer\_Phone\_Number, Customer\_Email, Customer\_Payment\_ID

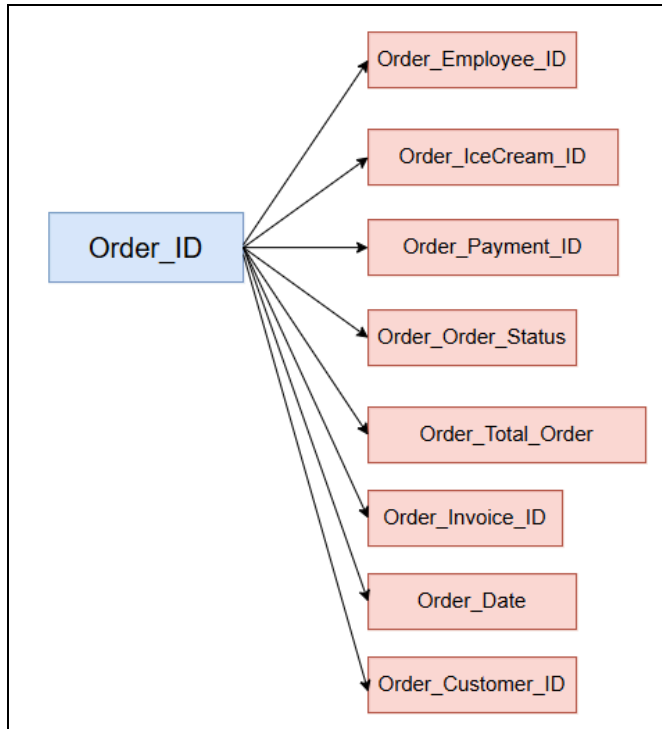
**Employee:**

{Employee\_ID} → Employee\_Name, Employee\_Birth\_Date, Employee\_Role, Employee\_Email, Employee\_Phone\_Number, Employee\_Salary, Employee\_Order\_ID



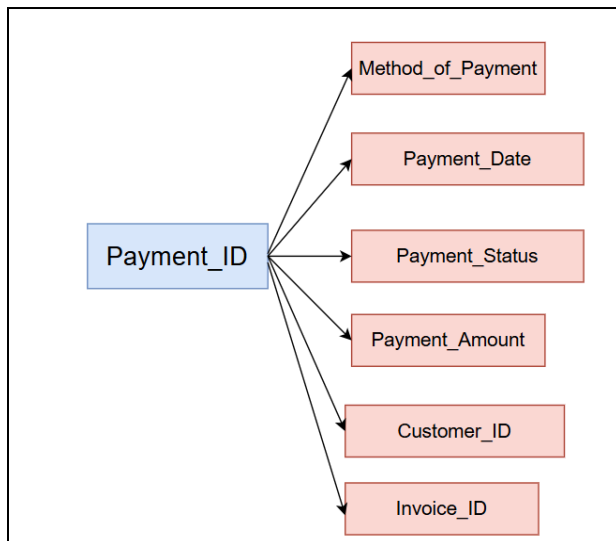
**Order\_IceCream:**

{Order\_ID} → Order\_Employee\_ID, Order\_IceCream\_ID, Order\_Payment\_ID,  
Order\_Order\_Status, Order\_Total\_Order, Order\_Invoice\_ID, Order\_Date, Order\_Customer\_ID



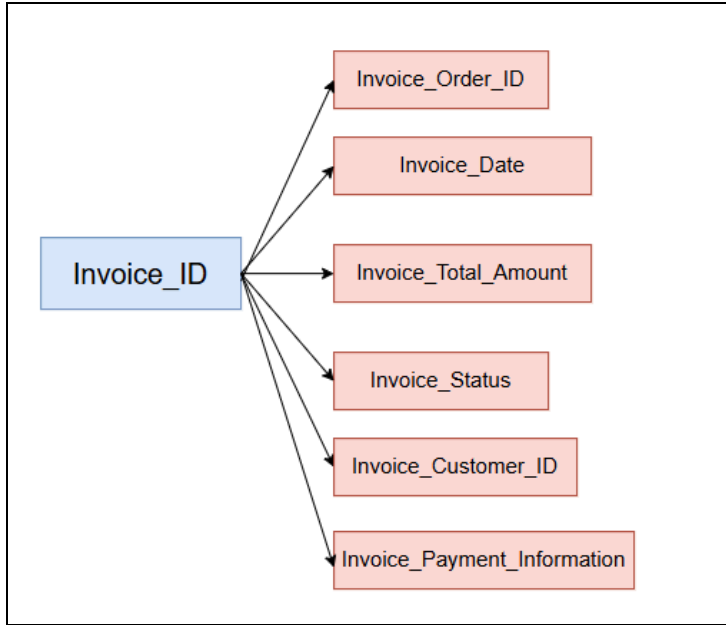
### PaymentInformation:

{Payment\_ID} → Method\_of\_Payment, Payment\_Date, Payment\_Status, Payment\_Amount, Customer\_ID, Invoice\_ID



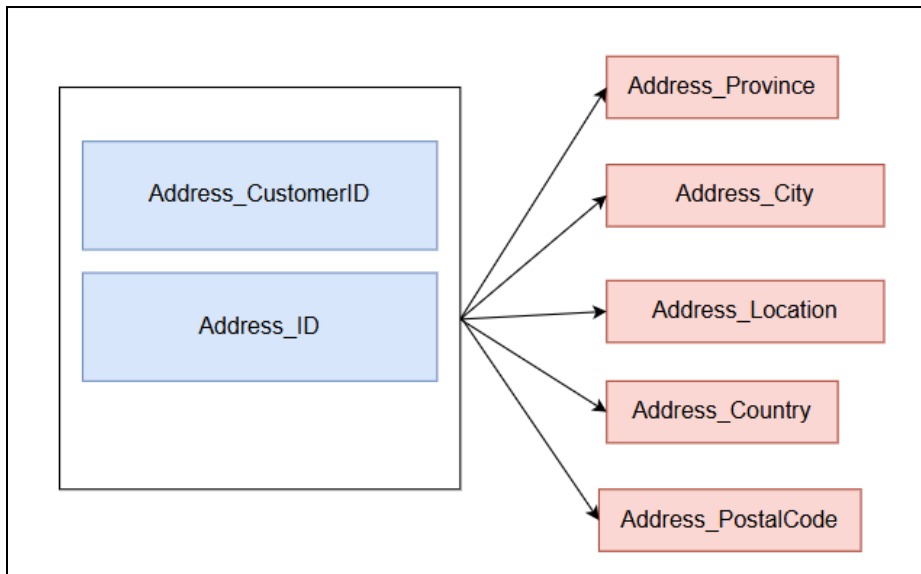
### Invoice:

{Invoice\_ID} → Invoice\_Order\_ID, Invoice\_Date, Invoice\_Total\_Amount, Invoice\_Status, Invoice\_Customer\_ID, Invoice\_Payment\_Information\_ID



**Address:**

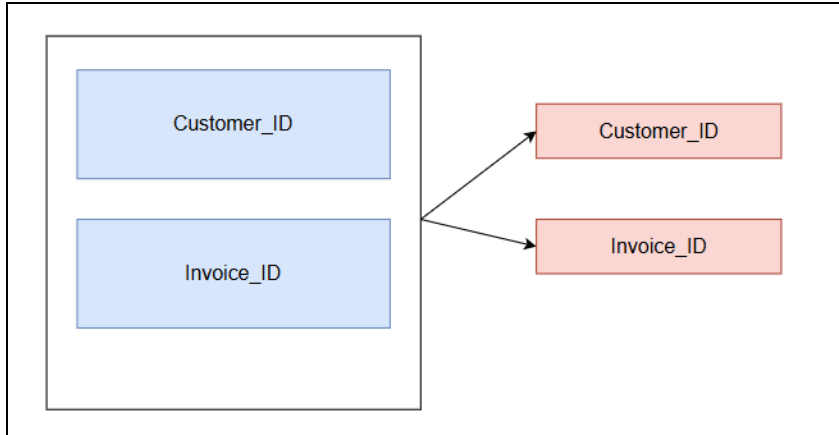
$\{\text{Address\_CustomerID}, \text{Address\_ID}\} \rightarrow \text{Address\_Province}, \text{Address\_City}, \text{Address\_Location}, \text{Address\_Country}, \text{Address\_PostalCode}$



**has:**

$\{\text{Customer\_ID}, \text{Invoice\_ID}\} \rightarrow \text{Customer\_ID}$

$\{\text{Customer\_ID}, \text{Invoice\_ID}\} \rightarrow \text{Invoice\_ID}$



## Assignment 8: Normalization 3NF/BCNF By Algorithm

Each table is 1NF as the table has one value assigned to it. The tables are 2NF since every attribute depends on the primary keys with no partial dependencies. The tables are 3NF because there are no transitive relationships and attributes can only depend on the table's primary key.

Each table is in BCNF because all of the attributes are dependent on the primary key.

### List of FDs:

Customer Table

- **Attributes:** Customer\_ID, Customer\_Name, Customer\_Phone, Customer\_Email
- **FDs:**
  - Customer\_ID → Customer\_Name, Customer\_Phone, Customer\_Email

**Explanation:** Customer\_ID is the primary key, and it determines all other attributes in the table.

Customer\_Address\_Payment Table

- **Attributes:** Customer\_ID, Customer\_Address\_ID, Customer\_Payment\_ID
- **FDs:**
  - Customer\_ID → Customer\_Address\_ID, Customer\_Payment\_ID

**Explanation:** Customer\_ID uniquely determines the associated address and payment details for each customer.

Order Table



- **Attributes:** Order\_ID, Order\_Date, Order\_Employee\_ID, Order\_IceCream\_ID
- **FDs:**
  - Order\_ID → Order\_Date, Order\_Employee\_ID, Order\_IceCream\_ID

**Explanation:** Order\_ID is the primary key and determines the order details.

#### Order\_Payment Table

- **Attributes:** Order\_ID, Payment\_ID, Invoice\_ID
- **FDs:**
  - Order\_ID → Payment\_ID, Invoice\_ID

**Explanation:** Order\_ID determines the associated payment and invoice IDs.

#### Payment Table

- **Attributes:** Payment\_ID, Payment\_Method, Payment\_Date, Payment\_Status, Payment\_Amount
- **FDs:**
  - Payment\_ID → Payment\_Method, Payment\_Date, Payment\_Status, Payment\_Amount

**Explanation:** Payment\_ID is the primary key and determines all payment details.

#### Customer\_Payment Table

- **Attributes:** Customer\_ID, Payment\_ID
- **FDs:**
  - Customer\_ID, Payment\_ID → Customer\_Payment

**Explanation:** The combination of Customer\_ID and Payment\_ID uniquely identifies the payment made by a specific customer.

#### Invoice Table

- **Attributes:** Invoice\_ID, Invoice\_Date, Invoice\_Amount
- **FDs:**
  - Invoice\_ID → Invoice\_Date, Invoice\_Amount

**Explanation:** Invoice\_ID determines the date and amount of the invoice.

#### Invoice\_Order Table

- **Attributes:** Invoice\_ID, Order\_ID

- **FDs:**
  - Invoice\_ID → Order\_ID

**Explanation:** Each Invoice\_ID is associated with a specific Order\_ID.

The PaymentInformation table will be used to conduct the 3NF and BCNF normalization algorithms. This is demonstrated as a way to show how to make a table 3NF/BCNF if violated. The attributes in the table are represented by acronyms to simplify the process, as shown in the legend below:

**PaymentInformation Table Legend:**

- Payment\_ID = PID
- Method\_of\_Payment = MOP
- Payment\_Date = PD
- Payment\_Status = PS
- Payment\_Amount = PA
- Customer\_ID = CID
- Invoice\_ID = IID

### 3NF Algorithm Implementation:

**Step 1: Find Functional Dependencies**

Functional Dependencies (FD) = {  
     PID → MOP, PD, PS, PA, CID, IID (1)  
     CID, IID → MOP, PS (2)  
     IID → PA (3)  
 }

In Dependency 1, PID determines all attributes, so there are no transitive dependencies. In Dependency 2, CID and IID are non-prime attributes that determine MOP and PS. Since PID determines both CID and IID (from Dependency 1), PID indirectly determines MOP and PS, creating a transitive dependency. In Dependency 3, IID determines PA and since IID is non-prime, PA is transitively dependent on PID.

**Step 2: Reduce List of Functional Dependencies**

| FD        | Closure Sets                            | Redundancy Status |
|-----------|-----------------------------------------|-------------------|
| PID → MOP | PID += {MOP, PID, PD, PS, PA, CID, IID} | Redundant         |
| PID → PD  | PID += {PID, MOP, PS, PA, CID, IID}     | Non-Redundant     |

|                            |                                               |               |
|----------------------------|-----------------------------------------------|---------------|
| $PID \rightarrow PS$       | $PID \neq \{PID, MOP, PS, PD, PA, CID, IID\}$ | Redundant     |
| $PID \rightarrow PA$       | $PID \neq \{PID, MOP, PD, PA, CID, IID\}$     | Redundant     |
| $PID \rightarrow CID$      | $PID \neq \{PID, MOP, PD, PA, IID\}$          | Non-Redundant |
| $PID \rightarrow IID$      | $PID \neq \{PID, MOP, PD, PA, CID\}$          | Non-Redundant |
| $CID, IID \rightarrow MOP$ | $CID, IID \neq \{CID, IID, PS\}$              | Non-Redundant |
| $CID, IID \rightarrow PS$  | $CID, IID \neq \{CID, IID, MOP\}$             | Non-Redundant |
| $IID \rightarrow PA$       | $IID \neq \{IID\}$                            | Non-Redundant |

After removing redundancies by separating the right hand side, the updated functional dependencies is:

Functional Dependencies (FD) = {  
 $PID \rightarrow PD, CID, IID$  (1)  
 $CID, IID \rightarrow MOP, PS$  (2)  
 $IID \rightarrow PA$  (3)  
 }

Now, partial dependencies need to be removed by minimizing the left hand side:

| FD                         | Closure Sets                             | Dependency Status                                               |
|----------------------------|------------------------------------------|-----------------------------------------------------------------|
| $CID, IID \rightarrow MOP$ | $CID \neq \{CID\}$<br>$IID \neq \{IID\}$ | Fully dependent because CID and IID alone cannot determine MOP. |
| $CID, IID \rightarrow PS$  | $CID \neq \{CID\}$<br>$IID \neq \{IID\}$ | Fully dependent because CID and IID alone cannot determine PS.  |

### Step 3: Find the Keys

Left Hand Side & Not Right Hand Side = {PID}. This means that PID is part of a candidate key.  
 Not Left Hand Side & Right Hand Side = {PD, MOP, PS, PA}. These non-prime attributes are not part of any candidate key.

### Step 4: Derive the final schema

**Relations:**

For R1 (PID, PD, CID, IID)

R2 (CID, IID, MOP, PS)

R3 (IID, PA)

There is a key in R1 (PID)

## BCNF Algorithm Implementation

**Relation:** R (PID, MOP, PD, PS, PA, CID, IID)

**Functional Dependencies:**

PID  $\rightarrow$  MOP, PD, PS, PA, CID, IID (1)

CID, IID  $\rightarrow$  MOP, PS (2)

IID  $\rightarrow$  PA (3)

First, we have to check if the left-hand sides are keys by making closures, as shown below.

PID  $\neq$  {PID, MOP, PD, PS, PA, CID, IID} is a key because its closure has all the attributes.

CID, IID  $\neq$  {CID, IID, MOP, PS} is not a key because its closure does not include all the attributes. This means that R is not BCNF with respect to CID, IID  $\rightarrow$  MOP, PS, so we have to decompose our relation in R11 and R12.

R11 = {PID, PD, PA, CID, IID}

**R12 = {CID, IID, MOP, PS} which is in BCNF**

Now, we have to check if R11 is in BCNF with FD11:

FD11 = {  
    PID  $\rightarrow$  PD, CID, IID  
    IID  $\rightarrow$  PA  
}

PID  $\neq$  {PID, PD, CID, IID, PA} it is a key

IID  $\neq$  {IID, PA} is not a key so split R11 to R111 and R112.

**R111 = {PID, PD, CID, IID} which is in BCNF.**

**R112 = {IID, PA} which is BCNF.**

Final BCNF Schema for R:

R1 = (CID, IID, MOP, PS)

R2 = (PID, PD, CID, IID)

R3 = (IID, PA)

## Assignment 9: User Interface of DBMS

```
Menu:
1. View All Tables
2. Create Table
3. Populate Tables
4. Update/Modify a Record
5. Remove a Record
6. Search in Records
7. Execute a Query
8. Commit Changes
9. Drop Tables
10. Exit

Enter your choice:
```

This terminal based program is connected to Oracle Database.

Functionality:

1. View all tables (and the records within the tables)

```
Enter your choice: 1
\Tables in the database:
1. PAYMENTINFORMATION
2. ORDER_ICECREAM
3. INVOICE
4. ICECREAM
5. HAS
6. FLAVOUR
7. EMPLOYEE
8. CUSTOMER
9. ADDRESS
10. Cancel

Enter the number of the table to view records: 1

Records in table: PAYMENTINFORMATION
PAYMENT_ID | METHOD_OF_PAYMENT | PAYMENT_DATE | PAYMENT_STATUS | PAYMENT_AMOUNT | CUSTOMER_ID | INVOICE_ID
-----
1. 401R | VISA | 2024-02-03 00:00:00 | DONE | 2443.0 | None | None
2. 402R | MasterCard | 2024-02-05 00:00:00 | PENDING | 1500.0 | None | None
3. 403R | PayPal | 2024-02-10 00:00:00 | FAILED | 300.5 | None | None
4. 404R | American Express | 2024-02-12 00:00:00 | DONE | 750.75 | None | None
5. 405R | Debit Card | 2024-02-15 00:00:00 | DONE | 120.0 | None | None
```

2. Create Table allows the users to create a table, with primary key, number of columns, and the column names.

```
Menu:
1. View All Tables
2. Create Table
3. Populate Tables
4. Update/Modify a Record
5. Remove a Record
6. Search in Records
7. Execute a Query
8. Commit Changes
9. Drop Tables
10. Exit

Enter your choice: 2

Enter the table name: newtesttable

Enter the primary key column name: primaryKey

Total Columns?? (including primary key)? 4

Enter the name for column 2: columnA

Enter the name for column 3: ColumnB

Enter the name for column 4: ColumnC
Table NEWTESTTABLE created successfully.
```

3. Populate table allows the user to add (and continue adding) values to the table.

```
Selected Table: NEWTESTTABLE
\Current Records:
The table is empty.

Enter values for the columns (blank to skip):

Enter value for PRIMARYKEY: firstRecordPrimaryKey
Enter value for COLUMNA: firstRecordColumnA
Enter value for COLUMNB: firstRecordColB
Enter value for COLUMNNC: firstRecColC
Inserted row into NEWTESTTABLE
(Dont forget to commit your changes!).

Do you want to add another record? (yes/no): yes

Enter value for PRIMARYKEY: 2ndkey
Enter value for COLUMNA: 2ndcola
Enter value for COLUMNB: 2ndcolb
Enter value for COLUMNNC: 2ndcolc
Inserted row into NEWTESTTABLE
(Dont forget to commit your changes!).

Do you want to add another record? (yes/no): no
```

4. Modify Record/ Update record: Allows the user to first select a table, then select the record (all using indexes) and then it prompts the user to enter the new value for the given record. If the user wishes to keep the record the same, then enter blank.

```

Enter your choice: 4
\Tables in the database:
1. PAYMENTINFORMATION
2. ORDER_ICECREAM
3. NEWTESTTABLE
4. INVOICE
5. ICECREAM
6. HAS
7. FLAVOUR
8. EMPLOYEE
9. CUSTOMER
10. ADDRESS
11. Cancel

Enter the number of the table to update records: 3

Records in table: NEWTESTTABLE
PRIMARYKEY | COLUMNNA | COLUMNNB | COLUMNNC
-----
1. firstRecordPrimaryKey | firstRecordColumnA | firstRecordColB | firstRecColC
2. 2ndkey | 2ndcola | 2ndcolb | 2ndcolc

Enter the number of the record to update (1-2) or 0 to cancel: 2

Enter new values for the fields (leave blank to keep current value):

PRIMARYKEY (current: 2ndkey):

COLUMNNA (current: 2ndcola):

COLUMNNB (current: 2ndcolb): 2ndcolbUPDATED

COLUMNNC (current: 2ndcolc):
Record updated in NEWTESTTABLE. Don't forget to commit!

Records in table: NEWTESTTABLE
PRIMARYKEY | COLUMNNA | COLUMNNB | COLUMNNC
-----
1. firstRecordPrimaryKey | firstRecordColumnA | firstRecordColB | firstRecColC
2. 2ndkey | 2ndcola | 2ndcolbUPDATED | 2ndcolc

```

5. Remove a record allows the user to remove a record from a selected table.

```
Enter your choice: 5
\Tables in the database:
1. PAYMENTINFORMATION
2. ORDER_ICECREAM
3. NEWTESTTABLE
4. INVOICE
5. ICECREAM
6. HAS
7. FLAVOUR
8. EMPLOYEE
9. CUSTOMER
10. ADDRESS
11. Cancel

Enter the number of the table to delete from: 3

Records in table: NEWTESTTABLE
PRIMARYKEY | COLUMN A | COLUMN B | COLUMN C
-----
1. firstRecordPrimaryKey | firstRecordColumnA | firstRecordColB | firstRecColC
2. 2ndkey | 2ndcola | 2ndcolbUPDATED | 2ndcolc

Enter the number of the record to delete (1-2) or 0 to cancel: 1
Record deleted from NEWTESTTABLE. Don't forget to commit!
```

```
Records in table: NEWTESTTABLE
PRIMARYKEY | COLUMN A | COLUMN B | COLUMN C
-----
1. 2ndkey | 2ndcola | 2ndcolbUPDATED | 2ndcolc
```

6. Search in records first prompts the user to enter the data value, and then it asks the user in which table they would like to search for the value. If the value exists then the program



will locate the entire record given just one value of the entire row.

```
Enter your choice: 6

Enter a search term: 2ndkey
\Tables in the database:
1. PAYMENTINFORMATION
2. ORDER_ICECREAM
3. NEWTESTTABLE
4. INVOICE
5. ICECREAM
6. HAS
7. FLAVOUR
8. EMPLOYEE
9. CUSTOMER
10. ADDRESS
11. Cancel

Select a table to search or Cancel to search all tables: 3
Found in table: NEWTESTTABLE
column: PRIMARYKEY
record number: 1
Record: 2ndkey | 2ndcola | 2ndcolbUPDATED | 2ndcolc
```

7. Allows the user to enter direct SQL queries into the program. To finish the query, the user must enter a blank and enter. Multi lined queries are accepted.

```
Enter your choice: 7
Enter your SQL query (type an empty line to finish):

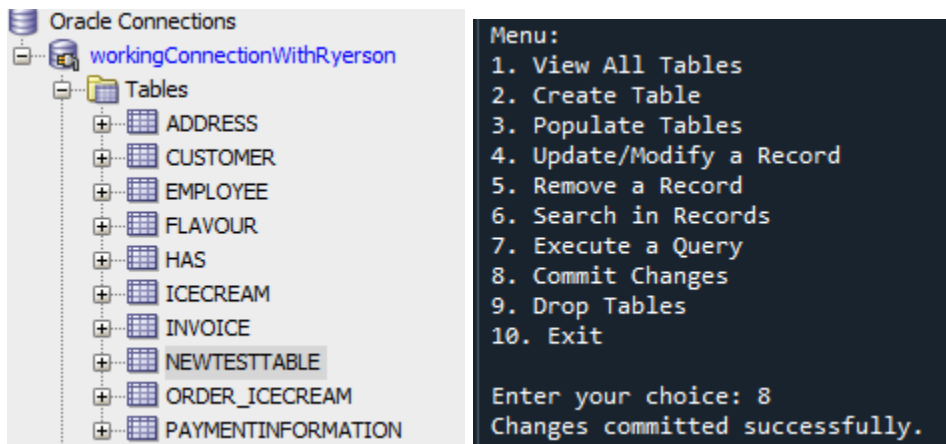
select * from newtesttable

Executing query: select * from newtesttable

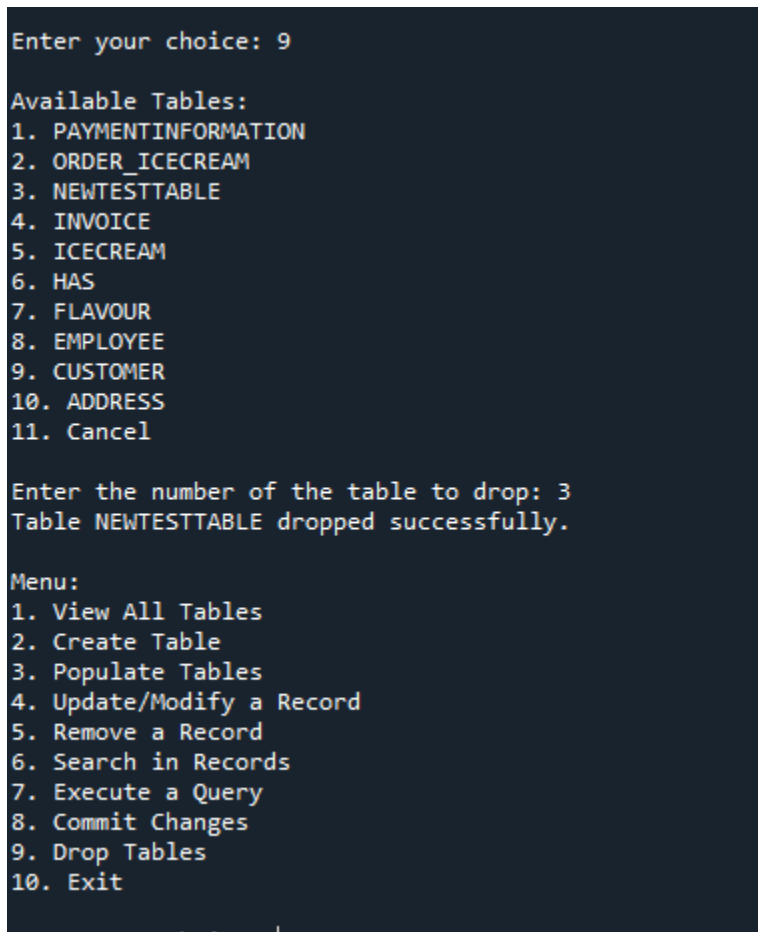
PRIMARYKEY | COLUMNNA | COLUMNNB | COLUMNNC
-----
2ndkey | 2ndcola | 2ndcolbUPDATED | 2ndcolc
```

8. Commit allows all the changes to get updated on the database. Exact same as commit on oracle SQL. As can be seen, the table we tested with, NEWTESTTABLE appears on

sqldeveloper once we use option 8 to commit changes.



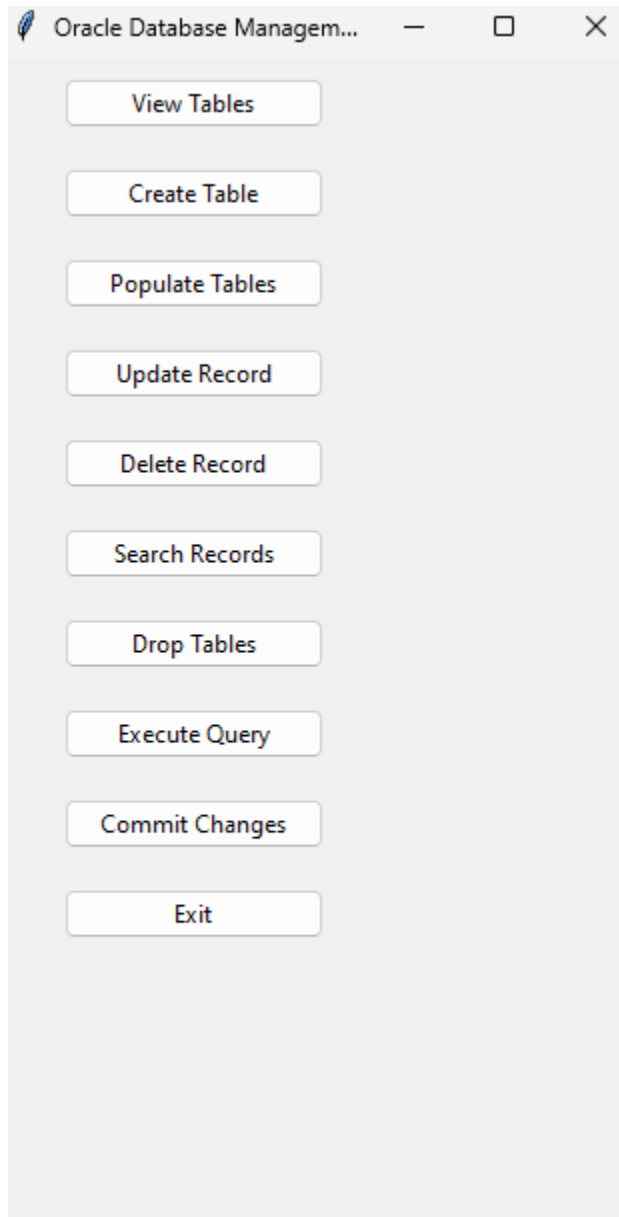
9. Drop table, DROP TABLE \_\_\_\_ CASCADE CONSTRAINTS PURGE. Note: You still have to commit your changes.



10. Exists the program, without committing.

The frontend UI is very similar to the backend terminal based program. Instead of asking the user to select options from the terminal, they now have some buttons to press.

The frontend UI was created using Python tkinter. The limitations of this UI is that the user still must enter the values in the terminal for their inputs, and receive the outputs in the terminal.



## Assignment 10: Relational Algebra

| Query                                                                                                                                                                                                                                                                                                                                                                                                                                            | RA Notation                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| <p><b>Simple Query 1:</b></p> <p><b>Explanation:</b> Displays all ice cream flavours that have no nuts.</p> <pre> SELECT DISTINCT   Flavour_ID as "Flavour_ID",   Flavour_Type as "Flavour Type" FROM Flavour WHERE Flavour_AllergyInfo = 'No nuts' </pre>                                                                                                                                                                                       | $\sigma_{FlavourAllergyInfo = 'No Nuts'}(Flavour)$ |
| <p><b>Simple Query 2:</b></p> <p><b>Explanation:</b> Displays the data of the staff from Employee ID.</p> <pre> SELECT   Employee_ID as "Employee ID",   Employee_Name as "Employee Name",   Employee_Birth_Date as "Birth Date",   Employee_Role as "Role",   Employee_Email as "Email",   Employee_Phone_Number as "Phone Number",   Employee_Salary as "Salary",   Employee_Order_ID as "Order_ID" FROM Employee ORDER BY Employee_ID; </pre> | $\sigma(Employee)$                                 |
| <p><b>Simple Query 3:</b></p>                                                                                                                                                                                                                                                                                                                                                                                                                    | $\sigma_{IceCreamStockQuantity \geq 700}(Flavour)$ |

|                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Explanation:</b> Displays all the number of ice creams that are greater than or equal to 700.</p> <pre>SELECT * FROM IceCream WHERE IceCream_StockQuantity &gt;= 700;</pre>                                                                                                                                                          |                                                                                                                                                            |
| <p><b>Simple Query 4:</b></p> <p><b>Explanation:</b> Displays all customers with active orders.</p> <pre>SELECT o.Order_ID,o.Order_Total_Order FROM Order_IceCream o WHERE o.Order_Order_Status IN ('Order Complete', 'Order Pending', 'Order Shipping') ORDER BY o.Order_ID ASC; --will show the orders in Order ID Ascending order</pre> | $\Pi_{o.Order\_ID, o.Order\_Total\_Order} \sigma_{o.Order\_Order\_Status = 'Order Complete' \vee 'Order Pending' \vee 'Order Shipping'} (Order\_IceCream)$ |
| <p><b>Simple Query 5:</b></p> <p><b>Explanation:</b> Display all invoices along with payment status.</p> <pre>SELECT DISTINCT Invoice_ID as "Invoice ID", Invoice_Total_Amount as "Total Amount", Invoice_Status as "Payment Status" FROM Invoice ORDER BY Invoice_ID;</pre>                                                               | $\Pi_{Invoice\_ID, Invoice\_Total\_Amount} \sigma (Order\_IceCream)$                                                                                       |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <p><b>Simple Query 6:</b></p> <p><b>Explanation:</b> Displays all employees who are a Production Operator.</p> <pre> SELECT     Employee_ID as "Employee ID",     Employee_Name as "Employee Name",     Employee_Salary as "Salary" FROM Employee WHERE Employee_Role = 'Production Operator' ORDER BY Employee_Name; </pre>                                                                                                                                                                                                         | $\Pi_{Employee\_ID, Employee\_Name, Employee\_Salary} \sigma_{Employee\_Role = 'Production Manager'} (Employee)$ |
| <p><b>Simple Query 7:</b></p> <p><b>Explanation:</b> Stores information about the customer's address. These customers include well-known stores such as Walmart, Shoppers Drug Mart, Frescho, Costco, and Nofrills.</p> <pre> SELECT     Address_CustomerID as "Customer ID",     Address_ID as "Address ID",     Address_Country as "Country",     Address_Location as "Location",     Address_Province as "Province",     Address_City as "City",     Address_PostalCode as "Postal Code" FROM Address ORDER BY Address_ID; </pre> | $\sigma(Address)$                                                                                                |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Simple Query 8:</b></p> <p><b>Explanation:</b> Displays the Customer ID that paid with VISA.</p> <pre> SELECT     Customer_ID as "Customer ID",     Invoice_ID as "Invoice ID",     Payment_ID as "Payment ID",     Method_of_Payment as "Payment Method",     Payment_Date as "Payment Date",     Payment_Amount as "Payment Amount",     Payment_Status as "Payment Status" FROM PaymentInformation WHERE Method_of_Payment = 'VISA' ORDER BY Payment_Date </pre> | $\Pi_{Customer\_ID, Invoice\_ID, Payment\_ID, Method\_of\_Payment, Payment\_Date, Payment\_Amount, Payment\_Status} \sigma_{Method\_of\_Payment = 'VISA'}(PaymentInformation)$ |
| <p><b>Simple Query 9:</b></p> <p><b>Explanation:</b> Provides information about the number of employees there are.</p> <pre> SELECT     Employee_Role as "Role",     COUNT(Employee_ID) as " Total Number of Employees" FROM Employee GROUP BY Employee_Role </pre>                                                                                                                                                                                                       | $Employee\_Role \overset{F}{COUNT}_{Employee\_ID} \sigma (Employee\_Role)$                                                                                                     |
| <p><b>Advanced Query 1:</b></p>                                                                                                                                                                                                                                                                                                                                                                                                                                           | $\Pi_{Flavour\_Type, IceCream\_PricePerUnit}$                                                                                                                                  |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <p><b>Explanation:</b> All flavours that cost more than the average price of all the ice creams.</p> <p>--All flavours that cost more than the average price of all the ice creams</p> <pre>SELECT f.Flavour_Type, ic.Icecream_PricePerUnit --ic is aliased in the next line FROM Flavour f JOIN IceCream ic ON f.Flavour_ID = ic.Icecream_FlavourID WHERE ic.Icecream_PricePerUnit &gt; ( SELECT AVG(Icecream_PricePerUnit) FROM IceCream );</pre>             | $\sigma_{IceCream\_PricePerUnit > AVG(IceCream\_PricePerUnit)}(Flavour \bowtie_{Flavour\_ID = Icecream\_Flavour} IceCream)$ |
| <p><b>Advanced Query 2:</b></p> <p><b>Explanation:</b> Top 3 flavours that contain the MOST amount of ingredients.</p> <p>--Top 3 flavours that contain the MOST amount of ingredients</p> <pre>SELECT * FROM ( SELECT f.Flavour_Type, LENGTH(f.Flavour_Ingredients) - LENGTH(REPLACE(f.Flavour_Ingredients, ',', '')) + 1 AS ingredientCount FROM Flavour f ORDER BY ingredientCount DESC ) WHERE ROWNUM &lt;= 3; --CHANGE THE 3 HERE TO GET MORE VALUES</pre> | $\Pi_{Flavour\_Type, ingredientCount} \sigma_{ROWNUM \leq 3}(Flavour)$                                                      |
| <p><b>Advanced Query 3:</b></p>                                                                                                                                                                                                                                                                                                                                                                                                                                 | $\Pi_{ic.Icecream\_ID, f.Flavour\_Type, ic.Icecream\_StockQuantity} \sigma_{ROWNUM \leq 3}(IceCream)$                       |



|                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <p><b>Explanation:</b> Top 3 ice creams with the highest amount of in stock quantity.</p> <pre> SELECT * FROM (     SELECT ic.Icecream_ID,     f.Flavour_Type,     ic.Icecream_StockQuantity     FROM IceCream ic     JOIN Flavour f ON     ic.Icecream_FlavourID = f.Flavour_ID     ORDER BY     ic.Icecream_StockQuantity DESC ) WHERE ROWNUM &lt;= 3; </pre>                                                                                  | $\bowtie_{Icecream\_FlavourID = Flavour\_ID} \text{Flavour})$                                                            |
| <p><b>Advanced Query 4:</b></p> <p><b>Explanation:</b> The price of each ice cream, and how much more or less it is than the average.</p> <pre> SELECT f.Flavour_Type, ic.Icecream_PricePerUnit, (ic.Icecream_PricePerUnit - (SELECT AVG(Icecream_PricePerUnit) FROM IceCream)) AS priceDifference --we can simply use the AVG function here to get the averages FROM IceCream ic JOIN Flavour f ON ic.Icecream_FlavourID = f.Flavour_ID; </pre> | $\Pi_{f.Flavour\_Type, ic.Icecream\_PricePerUnit} \sigma_{Icecream\_FlavourID = Flavour\_ID} (IceCream \bowtie Flavour)$ |
| <p><b>Advanced Query 5:</b></p> <p><b>Explanation:</b> The amount of salary is earned by each role.</p>                                                                                                                                                                                                                                                                                                                                          | $\Pi_{e.Employee\_Role, totalSalary} \textcolor{blue}{F}_{SUM\ e.Employee\_Role; SUM(Employee\_Role)} \sigma (Employee)$ |

|                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <p>-- how much salary is earned by each role?</p> <pre>SELECT e.Employee_Role, SUM(e.Employee_Salary) AS totalSalary FROM Employee e GROUP BY e.Employee_Role ORDER BY totalSalary ASC;</pre>                                                                                                                                                                                    |                                                                                          |
| <p><b>Advanced Query 6:</b></p> <p><b>Explanation:</b> Displays the employees that have not been assigned any orders.</p> <p>--which employees are slacking? i.e which employees have not been assigned any orders?</p> <pre>SELECT e.Employee_Name FROM Employee e WHERE NOT EXISTS (   SELECT 1   FROM Order_IceCream oi   WHERE oi.Order_Employee_ID = e.Employee_ID );</pre> | $\Pi_{e.Employee\_Name}(\text{Employee} - \Pi_{e.Employee\_ID}(\text{Order\_IceCream}))$ |
| <p><b>Advanced Query 7:</b></p> <p><b>Explanation:</b> TOTAL revenue generated by ALL payment methods.</p> <p>--TOTAL revenue generated by ALL payment method</p> <pre>SELECT pi.Method_Of_Payment, SUM(pi.Payment_Amount) AS totalRevenue FROM PaymentInformation pi GROUP BY pi.Method_Of_Payment</pre>                                                                        | $F_{pi.Method\_Of\_Payment; SUM(pi.Payment\_Amount)}(\text{PaymentInformation})$         |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ORDER BY totalRevenue ASC;                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                               |
| <p><b>Advanced Query 8:</b></p> <p><b>Explanation:</b> Customers with a relatively similar postal code, this is to determine which customers are located closer to each other.</p> <p>--CUSTOMERS with a relatively similiar postal code, this is to determine which customers are located closer to each other</p> <p>SELECT<br/> SUBSTR(a.Address_PostalCode, 1, 3)<br/> AS postalArea, c.Customer_Name<br/> FROM Address a<br/> JOIN Customer c ON<br/> a.Address_CustomerID = c.Customer_ID<br/> WHERE ROWNUM &lt;= 3<br/> ORDER BY<br/> SUBSTR(a.Address_PostalCode, 1, 3),<br/> c.Customer_Name;</p> | $\Pi_{postalArea, c.Customer\_Name}$ $(\sigma_{ROWNUM \leq 3}(\rho_{postalArea} \leftarrow SUBSTR(a.Address\_PostalCode)$ $(Address \bowtie_{Address\_CustomerID = Customer\_ID} Customer)))$ |
| <p><b>Advanced Query 9:</b></p> <p><b>Explanation:</b> Counts the number of customers per city and show their names separated by commas.</p> <p>--COUNT the number of customers per city and show their names separated by commas</p> <p>SELECT a.Address_City, COUNT(*) AS customerCount,<br/> LISTAGG(c.Customer_Name, ', ')<br/> WITHIN GROUP (ORDER BY<br/> c.Customer_Name) AS customerNames<br/> FROM Address a</p>                                                                                                                                                                                  | $F_{a.Address\_City, COUNT(*) \rightarrow customerCount, LISTAGG(c.Customer\_Name) \rightarrow customerNames}$ $(Address \bowtie_{a.Address\_CustomerID = Customer\_ID} Customer)$            |

|                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| JOIN Customer c ON<br>a.Address_CustomerID = c.Customer_ID<br>GROUP BY a.Address_City<br>ORDER BY customerCount DESC;                                                                                                                                                                                                                                                                                                              |                                                                                                                                                        |
| <b>Advanced Query 10:</b><br><br><b>Explanation:</b> Finding ALL orders that cost more than the average of all the orders.<br><br>--Finding ALL orders that cost more than the average of all the orders<br>SELECT oi.Order_ID,<br>oi.Order_Total_Order<br>FROM Order_IceCream oi<br>WHERE oi.Order_Total_Order > (<br>SELECT AVG(oi2.Order_Total_Order)<br>FROM Order_IceCream oi2<br>)<br>ORDER BY oi.Order_Total_Order<br>DESC; | $\Pi_{oi.Order\_ID, oi.Order\_Total\_Order} \sigma_{oi.Order\_Total\_Order > AVG}(\text{Order\_IceCream})$                                             |
| <b>Advanced Query 11:</b><br><br><b>Explanation:</b> Top three oldest UNPAID invoices, this will include the canceled invoices too.<br><br>--Top 3 oldest UNPAID invoices, this will include the canceled invoices too<br>SELECT *<br>FROM (<br>SELECT i.Invoice_ID, i.Invoice_Date,<br>i.Invoice_Status<br>FROM Invoice i<br>WHERE i.Invoice_Status != 'Paid'         )                                                           | $\sigma_{ROWNUM \leq 3}(\text{Sort}(\Pi_{i.Invoice\_ID, i.Invoice\_Date, i.Invoice\_Status}(\sigma_{i.Invoice\_Status \neq 'Paid'}(\text{Invoice}))))$ |

|                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| ORDER BY<br>TO_DATE(i.Invoice_Date,<br>'DD/MM/YYYY') ASC<br>)<br>WHERE ROWNUM <= 3;                                                                                                                                                                                                                                                                                                                                                       |                                                                                                      |
| <b>Advanced Query 12:</b><br><br><b>Explanation:</b> Total paid invoices for each month.<br><br>--Total paid invoices for each month<br>SELECT EXTRACT(MONTH FROM<br>TO_DATE(i.Invoice_Date,<br>'DD/MM/YYYY')) AS invoiceMonth,<br>SUM(i.Invoice_Total_Amount) AS<br>totalPaid<br>FROM Invoice i<br>WHERE i.Invoice_Status = 'Paid'<br>GROUP BY EXTRACT(MONTH FROM<br>TO_DATE(i.Invoice_Date,<br>'DD/MM/YYYY'))<br>ORDER BY invoiceMonth; | $F_{invoiceMonth; SUM(Invoice\_Total\_Amount)}(\sigma_{Invoice\_Status = 'Paid'}(Invoice))$          |
| <b>Interesting Query 1:</b><br><br><b>Explanation:</b> Displays all the customers and flavours<br>--Interesting Query 1: UNION<br>SELECT<br>Flavour_Type AS Name<br>FROM<br>Flavour<br>UNION<br>SELECT<br>Customer_Name AS Name<br>FROM<br>Customer;                                                                                                                                                                                      | $\Pi_{Flavour\_Type \rightarrow Name}(Flavour) \cup \Pi_{Customer\_Name \rightarrow Name}(Customer)$ |
| <b>Interesting Query 2:</b>                                                                                                                                                                                                                                                                                                                                                                                                               | $\Pi_{f.Flavour\_ID, f.Flavour\_Type}($                                                              |

|                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Explanation:</b> Displays flavours that might have Hazel nuts.</p> <p>-- Interesting Query 2: EXISTS<br/> -- Displays flavours that might have Hazel nuts.</p> <pre> SELECT DISTINCT     f.Flavour_ID AS "Flavour_ID",     f.Flavour_Type AS "Flavour Type" FROM Flavour f WHERE EXISTS (     SELECT 1     FROM Flavour f2     WHERE f2.Flavour_ID = f.Flavour_ID     AND f2.Flavour_AllergyInfo = 'May contain Hazel nuts' ); </pre> | $\sigma_{f.Flavour\_ID=f2.Flavour\_ID \wedge Flavour\_AllergyInfo = 'May contain Hazel nuts'}(Flavour)$                                      |
| <p><b>Interesting Query 3:</b></p> <p><b>Explanation:</b> Displays the invoices of every payment status and orders the results through descending order.</p> <pre> SELECT     Invoice_Status as "Payment Status",     COUNT(Invoice_ID) as "Number of Invoices" FROM Invoice GROUP BY Invoice_Status ORDER BY "Number of Invoices" DESC; </pre>                                                                                             | $F_{Invoice\_Status; COUNT(Invoice\_ID)}(Invoice)$                                                                                           |
| <p><b>Interesting Query 4:</b></p> <p><b>Explanation:</b> Displays the invoices based on payment status except for Paid</p>                                                                                                                                                                                                                                                                                                                 | $(F_{Invoice\_Status; COUNT(Invoice\_ID)}(Invoice)) - (F_{Invoice\_Status; COUNT(Invoice\_ID)}(\sigma_{Invoice\_Status = 'Paid'}(Invoice)))$ |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <pre>-- Interesting Query 4: MINUS -- Displays the invoices based on payment status except for Paid SELECT     Invoice_Status as "Payment Status",     COUNT(Invoice_ID) as "Number of Invoices" FROM Invoice GROUP BY Invoice_Status MINUS SELECT     Invoice_Status as "Payment Status",     COUNT(Invoice_ID) as "Number of Invoices" FROM Invoice WHERE Invoice_Status = 'Paid' GROUP BY Invoice_Status ORDER BY "Number of Invoices" DESC;</pre> |                                                                                                                                  |
| <p><b>Interesting Query 5:</b></p> <p><b>Explanation:</b> Displays The employees who do not have any orders assigned</p> <pre>-- Interesting Query 5: HAVING -- Displays The employees who do not have any orders assigned SELECT     e.Employee_Name FROM     Employee e LEFT JOIN     Order_IceCream oi ON e.Employee_ID = oi.Order_Employee_ID GROUP BY     e.Employee_Name HAVING</pre>                                                           | $\Pi_{e.Employee\_Name}(\sigma_{COUNT(oi.Order\_ID)(Employee \bowtie_{e.Employee\_ID = Order\_Employee\_ID} Order\_IceCream))})$ |

|                         |  |
|-------------------------|--|
| COUNT(oi.Order_ID) = 0; |  |
|-------------------------|--|

### **Conclusion:**

This Ice Cream Manufacturer Database Management System (DBMS) benefits users by allowing them to navigate and engage in all the data within the database. This DBMS helps users store information, such as the details of customers, flavours, payment information, employees, address, ice cream, and invoice. We were tasked to complete ten assignments throughout the semester of this course, and have acquired a lot of knowledge about databases and their way of operating. Oracle 12c was used to create the tables and queries of the database. This DBMS incorporates several fundamental concepts, which include ER diagrams, tables (drop, create and populate), queries (simple, advanced and interesting), views, functional dependencies, 3NF, BCNF and relational algebra notation.

We have created a Graphical User Interface (GUI) of our Ice Cream Manufacturer through Python to make it accessible and user-friendly for users to use. This GUI can provide users the ability to delete, edit and add information. This project was a meaningful experience as we were able to enhance our skills as a group through cooperation, teamwork and knowledge.



## References

- [1] Figueiredo, M.S.N., and A.M. Pereira. “Managing Knowledge – the Importance of Databases in the Scientific Production.” *Procedia Manufacturing*, vol. 12, 2017, pp. 166–173, <https://doi.org/10.1016/j.promfg.2017.08.021>.
  
- [2] Tsarouhas, Panagiotis. “Applied Sciences | Free Full-Text | Reliability, Availability, and Maintainability (RAM) Study of an Ice Cream Industry.” *Mdpi.com*, 2020, [www.mdpi.com/2076-3417/10/12/4265/review\\_report](http://www.mdpi.com/2076-3417/10/12/4265/review_report). Accessed 19 Sept. 2024.